Smart Forklift Localization System

Parshva Patel *parshva.patel@iitgn.ac.in*
Kush Kamlesh Mehta *kush.metha@iitgn.ac.in*
Dev Kaushik Panchal *dev.panchal@iitgn.ac.in*
Bhutaiya Dhrumil Pareshbhai *24110075@iitgn.ac.in*
Shah Rishabh Analbhai *24110327@iitgn.ac.in*
Dept. of Mechanical Engineering, Indian Institute of Technology Gandhinagar

Advisor: Prof. Madhu Vadali
Dept. of Mechanical Engineering, Indian Institute of Technology Gandhinagar

Abstract

Accurate indoor localisation is essential for safe and efficient autonomous vehicle operation in dynamic industrial environments where GPS is unavailable. This document presents a complete real-time localisation stack for a smart forklift, fusing Ultra-Wideband (UWB) ranging data with Inertial Measurement Unit (IMU) kinematics. To overcome multipath interference and power-dependent distance biases inherent to indoor UWB ranging, raw measurements are corrected via a 13-feature binned Gradient Boosted decision tree ensemble. For real-time execution on an embedded Raspberry Pi 5, the machine learning pipeline is compiled into an optimised ONNX runtime graph, reducing per-prediction latency to 0.035 ms and eliminating serial buffer overflows when scaling to a 40 Hz ranging cadence across four active anchors. A suite of geometric multilateration solvers handles variable anchor visibility, incorporating a motion-extrapolation heuristic for two-anchor disambiguation alongside a Geometric Dilution of Precision (GDOP) confidence metric to dynamically scale measurement variance. Finally, a five-state Extended Kalman Filter (EKF) operating at 100 Hz fuses high-frequency IMU predictions with UWB-derived position fixes. The architecture is deployed within a modular ROS 2 framework that wirelessly streams fused pose estimates over TCP to a remote monitoring display.

I. Introduction

A. The Concept of Localization in Robotics

Localization is one of the most fundamental and critical competencies in the field of robotics. In its simplest form, localization is the process by which a robot or autonomous system determines its current position and orientation within a specific environment. For a human, knowing one's location might involve recognizing landmarks or reading signs. For a robot, this process requires reading raw data from various hardware sensors and processing that data through complex algorithms to calculate precise coordinates relative to a known map or reference frame. Without a robust and accurate localization system, a robot is fundamentally "blind" to its place in the world, making it impossible to plan paths, navigate safely, or execute meaningful tasks.

B. Importance of Localization for Autonomous and Industrial Systems

In the context of modern industrial automation, such as smart factories and advanced warehouses, localization takes on an even greater level of importance. Autonomous vehicles and industrial robots, including automated guided vehicles (AGVs) and smart forklifts, operate in highly dynamic and densely populated environments. These spaces are often shared with human workers, manually operated machinery, and constantly shifting inventory.

Accurate localization in these environments is not merely a matter of efficiency; it is a critical safety requirement. If a smart forklift cannot accurately determine its position down to a matter of centimeters, it runs the risk of colliding with structural pillars, dropping heavy payloads, or, most severely, endangering human lives. Furthermore, modern warehouse management relies on the concept of Industry 4.0, where every asset is tracked in real-time. A highly accurate localization system allows a central fleet management software to route multiple forklifts simultaneously, preventing traffic bottlenecks, optimizing energy consumption, and ensuring that materials are transported exactly when and where they are needed.

C. Common Localization Techniques and the Indoor Challenge

There are several established techniques for localization, but they often face severe limitations when applied to indoor industrial environments. The Global Positioning System (GPS), for example, is the worldwide standard for outdoor navigation. However, GPS relies on weak satellite signals that cannot reliably penetrate the concrete and steel roofs of warehouses, rendering it completely useless for indoor robotics.

Other common indoor techniques include vision-based localization using cameras or Light Detection and Ranging (LiDAR) sensors. While highly accurate, cameras can fail in poor lighting conditions, and their performance is easily disrupted by changing visual environments – such as when a warehouse aisle is suddenly blocked by new pallets. LiDAR is robust but incredibly computationally expensive and significantly increases the overall cost of the hardware system.

Because of these limitations, alternative sensor architectures must be considered. For indoor environments where robustness, cost-effectiveness, and computational efficiency are paramount, a combination of Ultra-Wideband (UWB) wireless technology and

Inertial Measurement Units (IMUs) presents an ideal solution. This combination bypasses the line-of-sight visual requirements of cameras and the outdoor limitations of GPS, providing a highly reliable foundation for indoor localization.

D. Scope and Organization of this Document

This paper consolidates six previously separate pieces of project documentation into a single, unified reference for the Smart Forklift Localization System. Section II introduces the UWB and IMU sensor technologies at a conceptual level. Section III documents the physical hardware architecture and wiring. Section IV documents the ROS 2 system architecture, node graph, and data flow. Section V derives the geometric UWB multilateration solvers and the GDOP confidence signal. Section VI documents the machine-learning distance-correction model. Section VII documents the conversion of that model to ONNX for real-time deployment. Section VIII derives the five-state Extended Kalman Filter used for IMU/UWB fusion. Section IX is reserved for experimental results. Section X concludes.

II. UWB and IMU Sensing Technology

A. Understanding Ultra-Wideband Radio

Ultra-Wideband (UWB) is an advanced wireless communication protocol that operates using radio waves. However, unlike traditional radio technologies that transmit data over a narrow frequency band (such as standard FM radio or standard Wi-Fi), UWB transmits information across a very wide spectrum of frequencies simultaneously – typically greater than 500 Megahertz (MHz).

Instead of broadcasting a continuous wave of varying amplitude or frequency, UWB operates by emitting billions of extremely short, sharp radio pulses per second. These pulses are on the scale of nanoseconds (one billionth of a second). Because the pulses are so incredibly brief and spread across such a wide frequency band, UWB signals do not interfere with standard narrow-band communications, allowing UWB to coexist peacefully with other wireless networks in a busy factory environment.

B. Working Principle: Time-of-Flight (ToF)

The primary mechanism that allows UWB to determine distance is known as Time-of-Flight (ToF). The principle of ToF is grounded in physics, specifically the speed of light. Because radio waves travel at the speed of light (approximately 300,000 kilometers per second in a vacuum), the time it takes for a radio pulse to travel from one device to another can be measured to calculate the exact distance between them.

When a UWB device sends a pulse to another UWB device, it records the exact timestamp of transmission. The receiving device notes the exact time of arrival, processes the signal, and sends a reply back. By calculating the total time taken for the signal to make this round trip, minus the processing time at the receiving end, the system can determine the exact time the signal spent traveling through the air. Multiplying this travel time by the speed of light yields a highly accurate distance measurement.

C. High Positioning Accuracy and Environmental Robustness

The use of incredibly short nanosecond pulses gives UWB a distinct advantage: high positioning accuracy. Because the pulse is so brief, the system can pinpoint the exact moment of arrival with extreme precision. This allows UWB to measure distances with an accuracy of 10 to 30 centimeters.

Furthermore, these short pulses make UWB highly resistant to a phenomenon known as “multipath fading.” In a warehouse, radio waves bounce off metal shelves, concrete floors, and forklifts. With traditional continuous-wave radio, these bounced signals arrive at the receiver slightly later than the direct signal, overlapping and confusing the receiver. With UWB, the pulse is so short that the direct signal arrives and is processed before the reflected signals bounce back, allowing the receiver to easily ignore the echoes.

D. Advantages over Bluetooth, Wi-Fi, and GPS

When compared to other wireless technologies, UWB is vastly superior for indoor positioning. GPS, as previously mentioned, cannot penetrate indoor environments. Wi-Fi and Bluetooth are traditionally designed for data transfer, not distance measurement. When Wi-Fi or Bluetooth are used for localization, they typically rely on Received Signal Strength Indicator (RSSI) – estimating distance based on how “loud” or strong the signal is. Signal strength, however, is easily blocked by physical objects, bodies, or interference, leading to wildly inaccurate estimates (often off by several meters). UWB’s reliance on Time-of-Flight, rather than signal strength, completely circumvents this issue, providing reliable, centimeter-level precision.

E. Typical Range and Industrial Applications

A standard UWB localization system typically has a reliable operating range of 50 to 200 meters depending on the specific hardware, antennas, and environmental obstacles. This makes it perfect for large-scale industrial applications. In modern manufacturing, UWB is used for tracking valuable tools across an assembly line, monitoring the movement of personnel for safety protocols in hazardous zones, and, critically, acting as the primary positioning framework for autonomous robots and smart vehicles like the forklift developed in this project.

F. UWB Architecture: Anchors and Tags

To utilize UWB technology for localizing a moving vehicle, the hardware system must be divided into two distinct categories of devices: Anchors and Tags.

1) The Role of UWB Anchors

An Anchor is a stationary UWB device that is permanently mounted within the physical environment, such as high up on warehouse walls, ceiling trusses, or structural pillars. The defining characteristic of an Anchor is that its physical position never changes. During the initial setup of the localization system, the precise three-dimensional coordinates (X, Y, and Z) of every Anchor must be meticulously measured and recorded in the system's central map. Because these coordinates are fixed and known with high certainty, the Anchors act as the absolute reference points for the entire environment.

2) The Role of the UWB Tag

A Tag, conversely, is the mobile UWB component. In the context of this project, the Tag is securely mounted onto the moving Smart Forklift. Unlike the Anchors, the Tag does not natively know its own coordinates. Its sole purpose is to continuously broadcast signals to the surrounding Anchors and receive their replies. As the forklift drives through the environment, the onboard Tag communicates with multiple Anchors simultaneously, using the Time-of-Flight principle to constantly calculate the exact radial distance between itself and each individual Anchor.

3) The Principle of Trilateration

To calculate a 2D coordinate (X and Y position) on the warehouse floor, the Tag must communicate with a minimum of three Anchors. The mathematical concept used to convert these distances into a position is called trilateration. Imagine the Tag communicates with Anchor A and determines it is exactly 5 meters away. The Tag now knows it exists somewhere on the edge of a circle with a 5-meter radius centered on Anchor A. Next, the Tag measures its distance to Anchor B (e.g., 7 meters); the system draws a second circle around Anchor B, and the Tag must exist at one of the two points where these two circles intersect. Finally, a third measurement is taken from Anchor C, and all three circles will intersect at exactly one specific point – the precise, real-time location of the Tag, and by extension the forklift. This entire calculation happens many times per second to provide continuous tracking. (The full geometric treatment of this problem, including the two-, three-, and four-anchor cases actually implemented, is given in Section V.)

G. Introduction to the Inertial Measurement Unit (IMU)

While UWB provides excellent absolute positioning, it is an external system that requires communication with the environment. To create a truly robust localization system, the forklift must also be able to sense its own internal motion. This is achieved through an Inertial Measurement Unit (IMU): a microscopic, self-contained electronic sensor package that measures the specific forces, angular rates, and sometimes magnetic fields acting upon the body it is attached to.

1) Internal Components of the IMU

For the purposes of this Smart Forklift project, the IMU hardware relies on two fundamental micro-electromechanical systems (MEMS) sensors: the Accelerometer and the Gyroscope. The Accelerometer measures linear acceleration – whenever the forklift speeds up, slows down, or physically bumps into something, the accelerometer measures the force of that change in velocity across three axes (forward/backward, left/right, and up/down); it also constantly measures the downward pull of Earth's gravity, which helps the system determine which way is "down." The Gyroscope measures angular velocity, the rate of rotation around those same three axes; if the forklift turns its steering, swerves, or pivots in place, the gyroscope precisely records how fast the vehicle is rotating.

By taking the continuous stream of linear acceleration from the accelerometer and angular velocity from the gyroscope, the onboard computer can calculate the orientation (the heading or direction the forklift is facing) and track the physical motion of the vehicle over time.

2) The Necessity of Sensor Fusion: Why Combine UWB and IMU?

A critical question in robotics is why one would need both an IMU and a UWB system if both can be used to track movement. The answer lies in the complementary strengths and weaknesses of each hardware component.

An IMU is highly responsive and can output data hundreds of times per second (high frequency). However, calculating position solely from an IMU involves mathematically adding up every small movement over time (a process called dead reckoning). Because IMU sensors are inherently noisy, tiny errors in measurement quickly accumulate. Within a matter of seconds, an IMU alone will "drift," meaning its estimated position will drift further and further away from reality.

Conversely, a UWB system provides absolute positioning – it does not drift over time because it constantly checks its position against fixed Anchors. However, UWB signals can momentarily drop out if a large metal object blocks the signal, and UWB generally updates at a slower frequency compared to an IMU.

By combining the two – a process called Sensor Fusion – the system achieves the best of both worlds. The IMU provides smooth, high-speed tracking of the forklift's exact orientation and movement between UWB updates, while the UWB system acts as a constant anchor to reality, instantly correcting any accumulated drift from the IMU. This hardware synergy is what makes the Smart Forklift's localization highly reliable.

III. Hardware Architecture

A. The Main Processing Unit: Raspberry Pi 5

To handle the continuous influx of data from the UWB Tags and the IMU, and to run the complex mathematical algorithms re-

quired for sensor fusion, a robust computational brain is required. This project utilizes the Raspberry Pi 5, a Single-Board Computer (SBC) that contains all the essential components of a standard desktop computer – a processor, memory, and input/output ports – condensed onto a single, credit-card-sized circuit board.

The Raspberry Pi 5 was selected for this architecture due to its significant leap in computational power over previous generations. It features a high-speed multi-core processor capable of executing the real-time localization calculations without delay. Crucially for hardware integration, the Raspberry Pi features a General-Purpose Input/Output (GPIO) header, allowing the computer to interface directly with low-level electronic hardware, such as the IMU sensor, using direct serial communication protocols. Additionally, it offers multiple high-speed USB ports, essential for connecting external devices like the UWB Tag.

Beyond raw processing power, the Raspberry Pi 5 includes built-in wireless networking capabilities, supporting modern, high-speed Wi-Fi standards – a vital feature for mobile robotics, as it allows the onboard computer to transmit data wirelessly without being tethered by Ethernet cables. Because it requires very little power, runs a full Linux operating system, and offers immense flexibility, the Raspberry Pi 5 is the industry standard for prototyping and deploying embedded robotics systems.

B. Hardware Architecture of the Smart Forklift System

The system is intentionally divided into two distinct processing nodes, utilizing two separate Raspberry Pi 5 boards, to separate the heavy computation from the user visualization.

1) The Onboard Processing Unit (Forklift Mounted)

The first Raspberry Pi 5 acts as the "Onboard Processing Unit" and is physically mounted onto the moving Smart Forklift. This board is the core computational engine of the localization system. Physically wired to this onboard Raspberry Pi are the two primary sensing hardware components: the IMU sensor and a single UWB Tag. Every millisecond, the Raspberry Pi reads the physical motion data from the IMU; simultaneously, it receives the calculated distance measurements from the attached UWB Tag.

The onboard computer runs the sensor-fusion algorithms, mathematically merging the IMU motion data with the UWB distance data to compute a single, highly accurate position (X, Y) and orientation (heading angle) of the forklift. Once this real-time coordinate is calculated, the onboard unit packages the data and transmits it wirelessly over a network.

2) The User Interface Unit (Stationary Monitoring)

The second Raspberry Pi 5 functions as the "User Interface Unit." Unlike the onboard unit, this computer is not mounted on the vehicle; it is situated at a remote monitoring station where an operator can easily view an attached display monitor. This unit is configured to create a dedicated wireless hotspot – a local Wi-Fi network specifically for the forklift to connect to. Once the onboard Raspberry Pi joins this hotspot, it continuously streams the processed position and orientation coordinates to the User Interface Unit, which renders them onto a graphical User Interface (UI), visually representing the forklift as an icon moving smoothly across a digital map of the environment in real time.

3) Stationary Infrastructure: Anchor Placement

Surrounding this moving system is the fixed UWB infrastructure. Several UWB Anchors are securely mounted to fixed, known coordinates around the perimeter and interior of the operating environment. These devices require constant electrical power but remain entirely stationary throughout operation, serving strictly as the invisible, mathematical reference framework against which the moving Tag calculates its distances.

4) The Comprehensive Data Flow Workflow

First, physical movement occurs: as the forklift drives, the onboard IMU continuously senses linear acceleration and rotational velocity, sent instantly over wired connections directly to the Onboard Raspberry Pi 5. Concurrently, the UWB Tag mounted on the forklift emits radio pulses into the environment; the stationary UWB Anchors receive these pulses and reply, and the Tag calculates the exact Time-of-Flight for each Anchor, generating a list of distances passed via USB connection into the Onboard Raspberry Pi 5.

Inside the Onboard Raspberry Pi, the noisy UWB distance data and the drifting IMU motion data are fed into the localization processing algorithm, producing a clean, accurate, real-time coordinate of the forklift's exact location and heading. This processed localization data is then pushed to the Raspberry Pi's internal Wi-Fi radio and transmitted wirelessly to the dedicated hotspot created by the second Raspberry Pi, which immediately updates its graphical software to match the newly calculated physical coordinates.

5) Hardware Communication Protocols

At the lowest level, the IMU communicates with the Onboard Raspberry Pi using standard embedded hardware protocols (such as I^2C or SPI), which allow for extremely fast data transfer over short copper wires. The UWB Tag interfaces with the Onboard Raspberry Pi via a secure Serial/USB connection, ensuring that the critical distance data is received without interruption. Through the air, the UWB Tags and Anchors communicate using ultra-high-frequency radio pulses, while the two Raspberry Pi boards communicate over standard 2.4 GHz or 5 GHz Wi-Fi protocols (TCP/IP). By separating the hardware into these specific channels, the system ensures that massive amounts of data can flow simultaneously without bottlenecks.

6) Advantages of the Proposed Hardware Architecture

- **Modularity** – separating computation (Onboard Pi) from visualization (UI Pi) means the graphical interface can crash without the forklift losing its sense of location, ensuring system stability.
- **Scalability** – expanding the workspace simply requires mounting more fixed Anchors; the forklift's onboard hardware does not need to change.
- **Robustness** – the integration of UWB and IMU guarantees accurate indoor localization entirely immune to the visual disruptions that plague camera systems and the line-of-sight issues of traditional sensors.
- **Extensibility** – because the forklift possesses a highly accurate, real-time awareness of its own position and orientation, the hardware is fully prepared to support future integration of autonomous navigation, path planning, and obstacle avoidance features.

C. Hardware Connections and Integration

1) Ultra-Wideband (UWB) Hardware

The system architecture employs a VACUS ESP32-DW1000 board as the UWB Tag, securely mounted directly onto the forklift and physically connected to the onboard Raspberry Pi 5. To create the necessary reference framework for localization, multiple identical boards are configured to operate as UWB Anchors, installed at fixed, known coordinates throughout the operational environment. To ensure modularity and ease of placement, each Anchor is powered independently using a standard power bank. These Anchors remain completely stationary throughout the system's operation and serve as absolute reference points. During operation, the mobile Tag communicates wirelessly with all surrounding Anchors using Ultra-Wideband technology to facilitate ranging and accurate localization.

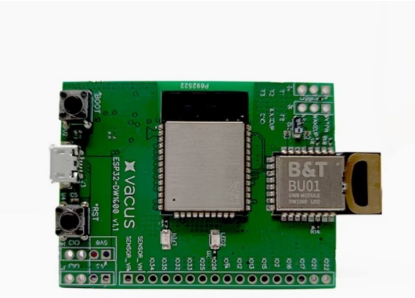


Figure 1: UWB hardware used as both Tag (mounted on the forklift) and Anchors (stationary reference nodes powered by power banks).

2) IMU to Raspberry Pi 5 Connection

Accurate motion estimation relies heavily on internal kinematic data. Although the specific IMU module depicted in Fig. 2 represents a different hardware revision than the one deployed in the final physical assembly, the underlying wiring configuration remains identical. The IMU communicates with the Raspberry Pi utilizing the I^2C (Inter-Integrated Circuit) communication protocol. The physical wiring connections are mapped explicitly as follows:

- **VCC** $\rightarrow$ 3.3 V (Power)
- **GND** $\rightarrow$ GND (Ground)
- **SDA** $\rightarrow$ GPIO 2 (SDA1)
- **SCL** $\rightarrow$ GPIO 3 (SCL1)

Through this dedicated I^2C interface, the Raspberry Pi continuously receives high-frequency acceleration and angular velocity measurements directly from the IMU, providing the raw data necessary for continuous motion estimation.

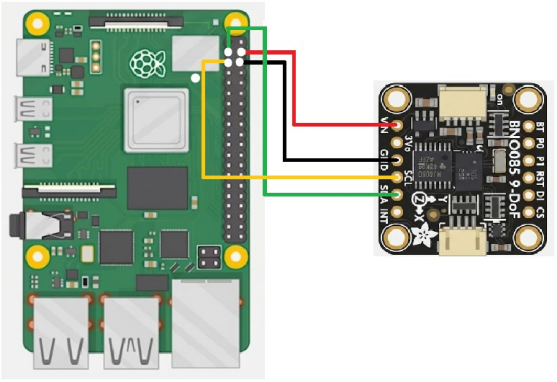


Figure 2: I^2C wiring connection between the IMU and the onboard Raspberry Pi 5.

3) Visualization and Display Interface

To provide immediate feedback to the operator, a dedicated display is integrated into the architecture: the official Raspberry Pi Display, connected directly to the Raspberry Pi 5 utilizing a standard high-speed display ribbon cable. This display unit serves as the primary visualization tool for the localization results, hosting the system's graphical user interface (GUI). As the Raspberry Pi processes the incoming sensor data, it actively renders the estimated real-time position and orientation of the smart forklift onto this screen.

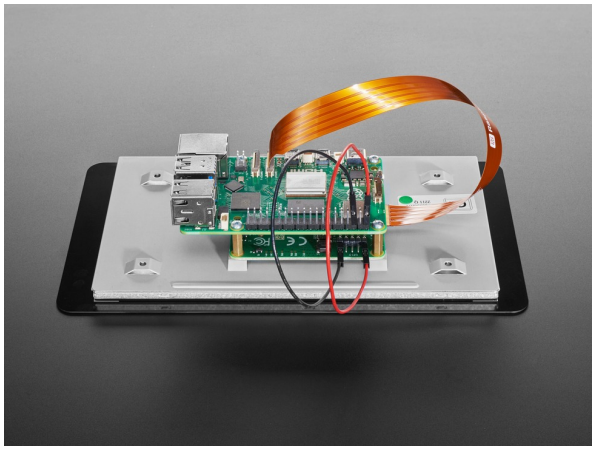


Figure 3: Raspberry Pi 5 connected to the Raspberry Pi Display for real-time visualization of localization data.

4) *Overall Hardware System Operation*

During active operation, the IMU – connected to the Raspberry Pi 5 via the I^2C interface – continuously measures and transmits internal inertial data. Simultaneously, the UWB Tag, physically connected to the same Raspberry Pi 5, communicates wirelessly with the array of stationary UWB Anchors, deployed at meticulously measured, known coordinates across the environment. The onboard Raspberry Pi acts as the central hub, simultaneously collecting the continuous stream of IMU motion measurements and the UWB ranging information. A sophisticated localization algorithm running locally on the Raspberry Pi actively fuses these two distinct sensor inputs; by combining the absolute positioning data from the UWB network with the precise, high-frequency motion data from the IMU, the algorithm accurately estimates the forklift's real-time position and orientation. Finally, these computed localization results are instantly transmitted to the connected Raspberry Pi Display, allowing operators to monitor the forklift's exact location and heading through the graphical user interface.

IV. ROS 2 System Architecture and Data Flow

A. Overview

The Smart Forklift Localization Stack fuses IMU (inertial) and UWB (Ultra-Wideband) sensor data through an Extended Kalman Filter (EKF) to determine the position and heading of the forklift in real time. The fused result is then streamed to a remote display over Wi-Fi so an operator can see the forklift's location and orientation. One component, `ml.py`, sits entirely outside ROS 2 and communicates with the `uwb_node` over a local UDP socket rather than a ROS topic.

> *In plain terms:* Four data sources feed into one fused position estimate: the IMU tells the system which way it's turning and accelerating, the UWB tags tell it roughly how far away it is from fixed anchors, and the EKF blends both into a single smooth, corrected pose – which is then broadcast wirelessly to a screen the operator can see.

B. System Architecture and Data Flow Diagram

Fig. 4 shows every component in the stack – hardware, the standalone `ml.py` process, all ROS 2 nodes, and the remote display – along with the exact topics, protocols, ports, message types, and rates connecting them.

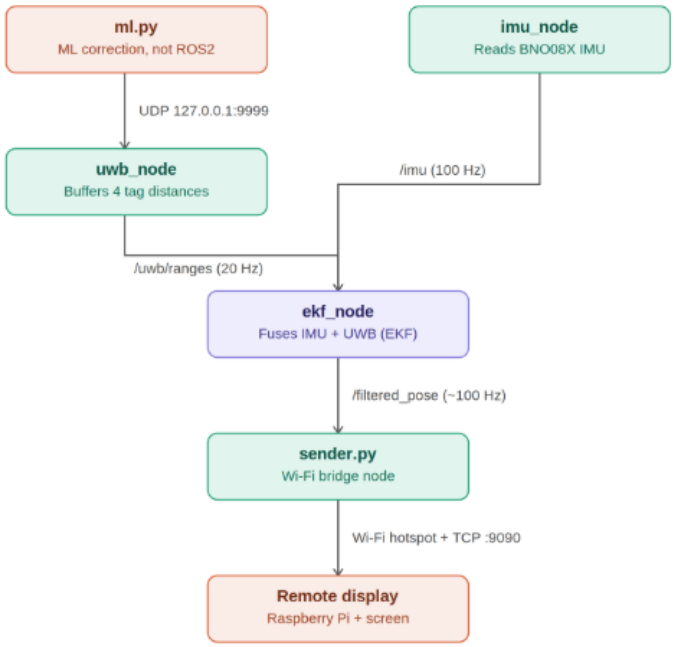


Figure 4: End-to-end data flow: sensors → `ml.py`/`imu_node` → `ekf_node` → `sender.py` → remote display.

C. System Components

1) `ml.py` — *ML Correction Backend (NOT a ROS 2 node)*

- Runs standalone (via `ExecuteProcess` in the launch file, in its own virtual environment).
- Reads raw ranging data from the UWB tag over serial (`/dev/ttyUSB0`).
- Extracts features, runs an ensemble of ONNX models to predict distance error.
- Applies an EMA (Exponential Moving Average) smoothing filter per tag.
- Sends corrected `{tag, dist}` JSON over UDP to `127.0.0.1:9999`.

Important: `ml.py` has no ROS topics in or out. It is completely invisible to the ROS 2 graph – its only interface to the rest of the system is a local UDP socket.

2) `imu_node` (ROS 2 node: `imu_node`)

- Reads orientation, acceleration, and angular velocity from a BNO08X IMU.
- Publishes: `/imu` (`sensor_msgs/Imu`) at 100 Hz.

3) `uwb_node` (ROS 2 node: `uwb_node`)

- Listens on UDP port 9999 for corrected distances sent by `ml.py`.
- Keeps a 4-slot buffer (tag IDs 1–4); marks a slot NaN if no fresh data arrived within 0.8 s (i.e., not in line-of-sight).
- Publishes: `/uwb/ranges` (`std_msgs/Float64MultiArray`, 4 values) at 20 Hz.

4) `ekf_node` (ROS 2 node: `unified_localization_node`)

- Subscribes: `/imu`, `/uwb/ranges`.
- Publishes: `/filtered_pose` (`geometry_msgs/PoseStamped`), `/uwb/active_anchor_count` (`std_msgs/Int32`).
- Maintains a 6-state EKF: $[x,\ y,\ z,\ v_x,\ v_y,\ \text{yaw}]$.
- On each IMU message: predicts motion.
- On each UWB message: geometrically solves position from however many anchors are visible (2–4) and fuses that into the filter, with gating and re-acquisition logic for dropped anchors.
- Publishes the fused pose at IMU rate ($\sim$100 Hz) and separately reports how many anchors are currently active.

5) `sender.py` (ROS 2 node: `wifi_bridge_node`)

- Subscribes: `/filtered_pose`, `/uwb/ranges`.
- On startup, creates a Wi-Fi hotspot (`smart_forklift`) and opens a raw TCP server on port 9090.
- Converts the pose quaternion to yaw (degrees), packages $[x, y, z, \text{yaw}, \texttt{active_uwb_list}]$ as JSON, and streams it over TCP at 10 Hz (reduced from `ekf_node`'s 100 Hz due to Raspberry Pi hardware and computational constraints) to a connected client (the remote Raspberry Pi + screen showing location/yaw in the UI).

Note on `active_uwb_list`: this list is derived from `/uwb/ranges` – it is the list of tag IDs (1–4) that are not NaN, i.e. currently in line-of-sight. It is computed inside `sender.py` itself (as `active_uwbs`), not read from `/uwb/active_anchor_count`. That topic is published separately by `ekf_node` and is not currently consumed by `sender.py`.

D. Data Flow Summary

Table I consolidates every link shown in the architecture diagram into a single reference.

Table 1: Inter-component data flow links

#	From → To	Channel	Message / Rate
1	UWB Tag → ml.py	Serial /dev/ttyUSB0	Raw ranging data, as received
2	ml.py → uwb_node	UDP 127.0.0.1:9999	JSON {tag, dist}, per reading
3	BNO08X → imu_node	HW driver	Orient./accel./ang. vel., 50 Hz
4	imu_node → ekf_node	/imu	sensor_msgs/Imu, 50 Hz
5	uwb_node → ekf_node	/uwb/ranges	Float64MultiArray[4], 20 Hz
6	uwb_node → sender.py	/uwb/ranges	Float64MultiArray[4], 20 Hz
7	ekf_node → sender.py	/filtered_pose	PoseStamped, $\sim$100 Hz
8	ekf_node (pub. only)	/uwb/active_anchor_count	Int32, $\sim$100 Hz
9	sender.py → Remote Pi + Screen	TCP :9090 over Wi-Fi	JSON [x,y,z,yaw,active_uwb_list], 10 Hz

E. System Bring-Up: The Launch File

The entire stack is brought up with a single command:

```
ros2 launch smart_forklift full_system.launch.py
```

This one launch file does two categorically different things.

1) One `ExecuteProcess` — `ml.py`

`ml.py` is not a ROS node, so it is launched as a raw OS subprocess (`ExecuteProcess`), deliberately using the Python interpreter from an isolated virtual environment (`~/ros2_ws/ml_env/bin/python3`) rather than the system Python. *Why an isolated venv:* this isolation exists so that `onnxruntime`, `scikit-learn`, and related ML libraries don't conflict with ROS 2's own Python environment.

2) Five Node Actions

The remaining five components are declared as standard ROS 2 `Node` actions, each with `package='smart_forklift'`, `output='screen'` (so logs print directly to the terminal instead of only going to log files), and matching executable/node names (Table II).

Table 2: ROS 2 node actions declared in the launch file

#	Node name	Executable	Package
1	uwb_node	uwb_node	smart_forklift
2	imu_node	imu_node	smart_forklift
3	encoder_node	encoder_node	smart_forklift
4	ekf_node	unified_localization_node	smart_forklift
5	sender	wifi_bridge_node	smart_forklift

V. Geometric Localization Solvers for UWB Range-Based Positioning

This section covers the geometric localization layer of the UWB positioning pipeline: turning raw, noisy range measurements between the moving tag and fixed anchors into a 2D position, plus a geometry-based confidence estimate that feeds into the Extended Kalman Filter (EKF) of Section VIII. Three solvers are built, one for each anchor-visibility case that shows up in a real deployment: a three-anchor nonlinear least-squares (NLLS) trilateration solver (§V-C), a four-anchor-or-more redundant solver built by running the three-anchor solver on multiple subsets and combining the results robustly (§V-D), and a two-anchor closed-form solver with a motion-based rule to pick between two possible solutions (§V-E). Geometric Dilution of Precision (GDOP) is derived as a geometry-only confidence signal (§V-F), and the full dispatch pipeline is given in §V-G.

A. UWB Hardware Platform and Ranging Cadence

1) Ranging Hardware and Protocol

Range measurements come from a UWB ranging layer built on the VACUS ESP32-DW1000 board: one mobile tag, up to four fixed anchors, each running a Decawave DW1000 radio over SPI via the `DW1000Ranging` Arduino library. To avoid needing tight clock sync (as one-way Time-of-Arrival systems do), the system uses Two-Way Ranging, specifically Asymmetric Double-Sided TWR via `computeRangeAsymmetric()`. Each ranging cycle is a four-message exchange:

$$\text{Tag} \xrightarrow{\text{POLL}} \text{Anchor}_i \xrightarrow{\text{POLL_ACK}} \text{Tag}$$
$$\xrightarrow{\text{RANGE}} \text{Anchor}_i \xrightarrow{\text{RANGE_REPORT}} \text{Tag}$$

Three firmware changes were made on top of the default library behavior:

1. **No software filtering**: `useRangeFilter(false)` is set so ρ_i is the raw single-cycle measurement, not a smoothed value that would distort what the EKF sees.
2. **Antenna delay calibration**: A_{delay} is tuned per board against a known baseline (instead of the default 16384) to remove systematic bias.
3. **CIR diagnostics**: the `RANGE_REPORT` payload is extended with raw Channel Impulse Response power readings – total power (`rx_power`), first-path power (`fp_power`), and their difference ΔdBm – used as the main non-line-of-sight (NLOS) indicator downstream.

2) Signal Modality

The DW1000 uses Impulse-Radio UWB (IR-UWB), per IEEE 802.15.4a-2007: instead of a narrowband carrier, it sends subnanosecond pulses spread over 500+ MHz (in the 3.5–6.5 GHz band). This gives fine time resolution, so individual multipath components can be told apart instead of blurring together – which is what makes the NLOS feature work: `fp_power` captures just the direct path, `rx_power` captures everything including reflections, and $\Delta\text{dBm} = \texttt{rx_power} - \texttt{fp_power}$ measures how much extra delayed energy there is from obstruction or reflection. The firmware runs in `MODE_LONGDATA_RANGE_LOWPOWER`, trading more airtime per exchange for better range and lower power – good for indoor tracking.

3) Multi-Anchor Cadence

Since the DW1000 is half-duplex, the tag cannot range with all anchors at once – it round-robins through the device table (`_networkDevices[]`), doing a full exchange with each anchor in turn (up to `MAX_DEVICES = 4`). Two parameters set the update rate: reply delay (7000 μs, the turnaround time for the responding node) and timer interval (80 ms, how often the tag starts a new poll). Since one exchange easily fits inside 80 ms, the timer is what actually paces things, giving a fixed aggregate output rate of about 12.5 Hz no matter how many anchors are active – but that bandwidth gets split across anchors, so per-anchor rate drops as $1/n$ (Table III).

Table 3: Sampling rate vs. anchor count

n	Pattern	Per-Anchor	Aggregate	Solver
1	A1 only	12.50 Hz	12.5 Hz	Range-only
2	A1,A2 alt.	6.25 Hz	12.5 Hz	Two-anchor (§V-E)
3	A1→A2→A3	4.17 Hz	12.5 Hz	Three-anchor (§V-C)
4	A1→..→A4	3.13 Hz	12.5 Hz	Four-anchor (§V-D)

Because polling is sequential, the ranges going into a multi-anchor fix are asynchronous snapshots taken over the previous $n \times 80$ ms cycle. That staleness is exactly why the pipeline needs validity gates and motion-continuity checks.

B. Range-Based Localization Model

1) State and Anchor Geometry

The tag's height is fixed to a trusted external value $h = \texttt{TRUSTED_Z}$, so the state to estimate is just $\mathbf{p} = [x \;\; y]^\top \in \mathbb{R}^2$. There are n active anchors at known positions $\mathbf{a}_i = [x_i, y_i, z_i]^\top$, surveyed in advance.

2) Forward Model and Residual

The true distance to anchor i is

$$d_i(\mathbf{p}) = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (h - z_i)^2}. \tag{1}$$

Each measured range is that distance plus noise,

$$\rho_i = d_i(\mathbf{p}) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_i^2), \tag{2}$$

assumed i.i.d. across anchors. The residual is

$$r_i(\mathbf{p}) = d_i(\mathbf{p}) - \rho_i. \tag{3}$$

3) Least-Squares Cost

The position estimate minimizes the sum of squared residuals:

$$J(\mathbf{p}) = \sum_{i=1}^{n} r_i(\mathbf{p})^2, \qquad \hat{\mathbf{p}} = \arg\min_{\mathbf{p}} J(\mathbf{p}). \tag{4}$$

Under the i.i.d. Gaussian noise model in (2), this is exactly the maximum-likelihood estimator – the negative log-likelihood reduces to (4) once terms that don't depend on $\mathbf{p}$ are dropped. The catch: there's no per-anchor weighting, so one anchor with a large NLOS bias just pulls $\hat{\mathbf{p}}$ off directly, with nothing to down-weight it.

4) Why Iteration Is Needed

$d_i(\mathbf{p})$ has a square root in it, so (4) is nonlinear and there's no closed-form $\hat{\mathbf{p}} = (A^\top A)^{-1} A^\top b$ solution like a linear model would have. That is why §V-C uses an iterative Levenberg–Marquardt solve for $n = 3$, extended to $n \geq 4$ in §V-D. For $n = 2$ (§V-E) there is an exact closed form instead.

C. Three-Anchor Solver: Levenberg–Marquardt NLLS

1) Idea

With three non-collinear anchors and no noise, three circles meet at one point and $J(\hat{\mathbf{p}}) = 0$. With noise, the circles don't quite meet, but $J(\mathbf{p})$ still has a clear minimum nearby. Since J is nonlinear, that minimum has to be found iteratively: linearize $d_i(\mathbf{p})$ at the current guess, solve the resulting linear least-squares step, move, repeat.

2) Linearization and Jacobian

Taylor-expanding the residual vector around the current estimate $\mathbf{p}^{(k)}$:

$$\mathbf{r}\left(\mathbf{p}^{(k)} + \boldsymbol{\delta}\right) \approx \mathbf{r}\left(\mathbf{p}^{(k)}\right) + \mathbf{J}\left(\mathbf{p}^{(k)}\right) \boldsymbol{\delta}, \tag{5}$$

where $\boldsymbol{\delta} = [\Delta x, \Delta y]^\top$. Since ρ_i is constant, $\partial r_i / \partial \mathbf{p} = \partial d_i / \partial \mathbf{p}$, giving

$$\frac{\partial d_i}{\partial x} = \frac{x - x_i}{d_i(\mathbf{p})}, \qquad \frac{\partial d_i}{\partial y} = \frac{y - y_i}{d_i(\mathbf{p})}. \tag{6}$$

Stacked over all anchors:

$$\mathbf{J}(\mathbf{p}) = \begin{bmatrix} \dfrac{x - x_1}{d_1(\mathbf{p})} & \dfrac{y - y_1}{d_1(\mathbf{p})} \\ \vdots & \vdots \\ \dfrac{x - x_n}{d_n(\mathbf{p})} & \dfrac{y - y_n}{d_n(\mathbf{p})} \end{bmatrix}. \tag{7}$$

Each row is just the unit vector from anchor i to the current estimate – this comes back in the GDOP derivation later.

3) Gauss–Newton Step

Plugging (5) into (4) and minimizing over $\boldsymbol{\delta}$ gives a linear least-squares problem. Setting the gradient to zero:

$$\mathbf{J}^\top \mathbf{J}\, \boldsymbol{\delta} = -\mathbf{J}^\top \mathbf{r}\left(\mathbf{p}^{(k)}\right). \tag{8}$$

When $\mathbf{J}^\top \mathbf{J}$ is invertible (true whenever anchors aren't collinear):

$$\boldsymbol{\delta} = -\left(\mathbf{J}^\top \mathbf{J}\right)^{-1} \mathbf{J}^\top \mathbf{r}\left(\mathbf{p}^{(k)}\right), \qquad \mathbf{p}^{(k+1)} = \mathbf{p}^{(k)} + \boldsymbol{\delta}. \tag{9}$$

4) Levenberg–Marquardt Damping

Plain Gauss–Newton can overshoot or diverge far from the optimum, since the linear approximation only holds locally. The Levenberg–Marquardt fix damps the step so it blends between the Gauss–Newton direction and gradient descent:

$$\left(\mathbf{J}^\top \mathbf{J} + \lambda \operatorname{diag}\left(\mathbf{J}^\top \mathbf{J}\right)\right) \boldsymbol{\delta} = -\mathbf{J}^\top \mathbf{r}\left(\mathbf{p}^{(k)}\right), \tag{10}$$

where λ is decreased after a successful step and increased after a rejected one – small λ recovers Gauss–Newton, large λ shrinks the step toward gradient descent. This is exactly what `scipy.optimize.least_squares(..., method='lm')` runs internally to solve (10), with the Jacobian approximated by finite differences rather than the analytic form in (7) (see §V-G, proposed improvement).

5) Existence, Uniqueness, Conditioning

For $n \geq 3$ non-collinear anchors, the minimum is locally unique whenever $\mathbf{J}^\top \mathbf{J}$ is full rank. That fails exactly when anchors are collinear: the columns of (7) become scalar multiples of each other, and position becomes unobservable perpendicular to the anchor line. This is the same conditioning issue behind GDOP (§V-F) – near-collinear anchors make $\mathbf{J}^\top \mathbf{J}$ ill-conditioned and amplify noise into large position error.

6) Multi-Start Seeding

$J(\mathbf{p})$ isn't convex, so Levenberg–Marquardt only finds the minimum closest to wherever it starts. To deal with this, the solver runs from multiple seeds: the last fused position, the anchor-field centroid, and a grid of points over the anchor bounding box (spacing `SEED_STEP`). Each seed gets its own LM run, and the winner is whichever converges to the lowest mean absolute residual:

$$\hat{\mathbf{p}} = \arg\min_{\mathbf{p}(x_0)} \frac{1}{3} \sum_{i=1}^{3} \left| d_i(\mathbf{p}(x_0)) - \rho_i \right|. \tag{11}$$

Note (11) is a different scale (meters of range error) than the sum-of-squares $J(\mathbf{p})$ that LM actually optimizes – it's just used to compare and gate across seeds. If (11) stays above `SOLVER_RESIDUAL_THRESH` for every seed, the solver returns nothing for that cycle and the EKF prediction takes over. Multi-start restarts are a standard, cheap fix for local minima in position-location problems, and in practice make this solver behave as if globally convergent for realistic UWB noise and geometry.

7) Worked Example

Anchors at height $z_i = 1.0\,\text{m}$: $A_1 = (0, 4, 1)$, $A_2 = (4, 0, 1)$, $A_3 = (-4, 0, 1)$, tag height $h = 0.5\,\text{m}$. True tag position $\mathbf{p}_{\text{true}} = (1, 1)$. Synthesized ranges from (1): $\rho_1 = \rho_2 = \sqrt{10.25} \approx 3.2015\,\text{m}$, $\rho_3 = \sqrt{26.25} \approx 5.1235\,\text{m}$.

Starting from the centroid $\mathbf{p}^{(0)} = (0, 0)$: all three distances equal $\sqrt{16.25} \approx 4.0311\,\text{m}$, giving residuals $r_1 = r_2 = 0.8296$, $r_3 = -1.0924$ and $J(\mathbf{p}^{(0)}) \approx 2.5697$.

The Jacobian at $\mathbf{p}^{(0)}$ works out to

$$\mathbf{J} = \begin{bmatrix} 0 & -0.9923 \\ -0.9923 & 0 \\ 0.9923 & 0 \end{bmatrix},$$

$$\mathbf{J}^\top \mathbf{J} = \begin{bmatrix} 1.9694 & 0 \\ 0 & 0.9847 \end{bmatrix},$$

$$\mathbf{J}^\top \mathbf{r} = \begin{bmatrix} -1.9072 \\ -0.8233 \end{bmatrix}.$$

Solving (9) (undamped, $\lambda = 0$) gives $\boldsymbol{\delta}^{(0)} = (0.9687, 0.8361)$, so $\mathbf{p}^{(1)} = (0.9687, 0.8361)$ – already close to the truth. Repeating the same update with LM damping annealed toward zero converges in a few more steps to $\hat{\mathbf{p}} = (1.0000, 1.0000)$, $J(\hat{\mathbf{p}}) \approx 0$, matching $\mathbf{p}_{\text{true}}$ exactly, as expected with no noise. Mean absolute residual (11) is 0, well under threshold, so the fix is accepted.

8) Assumptions, Advantages, Limitations

Assumptions: noise is zero-mean, roughly Gaussian, i.i.d. (no per-anchor weighting); anchors aren't collinear; h and z_i are accurate, since a bias in either shows up directly as horizontal position bias.

Advantages: maximum-likelihood optimal under the Gaussian model; degrades gracefully under moderate noise since it minimizes a smooth cost rather than needing an exact solution; multi-start seeding gives good practical robustness against non-convexity, cheaply.

Limitations: no formal global-convergence guarantee; no robust loss or per-anchor weighting, so one bad NLOS range pulls the fix directly (this is exactly what motivates the four-anchor solver in §V-D); near-collinear geometry means large GDOP and noise amplification.

Complexity: each LM iteration costs $O(n)$ (Jacobian) plus $O(1)$ (2×2 solve), so total cost is $O(S \cdot n)$ across S seeds.

D. Four-(or-More)-Anchor Solver: Redundant Subsets + Median Combination

1) Formulation

With $n = 4$ anchors there are $\binom{4}{3} = 4$ three-anchor subsets $S_1, \ldots, S_4$. Each gets solved independently by the three-anchor solver, giving either an accepted candidate $\hat{\mathbf{p}}_j$ or nothing. Let $\{\hat{\mathbf{p}}_j\}_{j \in \mathcal{J}}$ be the accepted set. If $|\mathcal{J}| \geq 3$, take the component-wise median (median of x's and median of y's separately – not the same as any actual $\hat{\mathbf{p}}_j$):

$$\mathbf{p}_{\text{med}} = (\text{median}_j(x_j),\, \text{median}_j(y_j))\,. \tag{12}$$

Keep whatever is within distance $\tau = \texttt{AVG3_OUTLIER_DIST_M}$ of the median:

$$\mathcal{K} = \{\hat{\mathbf{p}}_j : \|\hat{\mathbf{p}}_j - \mathbf{p}_{\text{med}}\| \leq \tau\}\,, \tag{13}$$

and average the survivors:

$$\hat{\mathbf{p}}_{\text{final}} = \frac{1}{|\mathcal{K}|} \sum_{\hat{\mathbf{p}}_j \in \mathcal{K}} \hat{\mathbf{p}}_j. \tag{14}$$

If fewer than three subsets converge, the median step is skipped and (14) is applied directly to whatever's in $\mathcal{J}$; if $\mathcal{J} = \varnothing$, no fix comes out this cycle.

2) Worked Example

Same four-anchor layout as before, but A_4 has an NLOS bias: reports $\rho_4 = 9.0\,\text{m}$ instead of its true $\approx 5.12\,\text{m}$.

Subset	Anchors	Bad A4?	Estimate $\hat{\mathbf{p}}_j$
S_1	A_1, A_2, A_3	No	$(1.0, 1.0)$
S_2	A_1, A_2, A_4	Yes	$(1.5, -1.2)$
S_3	A_1, A_3, A_4	Yes	$(-0.5, -0.8)$
S_4	A_2, A_3, A_4	Yes	$(0.2, -1.5)$

Median: x's sorted $\{-0.5, 0.2, 1.0, 1.5\} \to 0.6$; y's sorted $\{-1.5, -1.2, -0.8, 1.0\} \to -1.0$. So $\mathbf{p}_{\text{med}} = (0.6, -1.0)$. Distances to median, with $\tau = 1.0\,\text{m}$: $S_1 \approx 2.04$ (rejected), $S_2 \approx 0.92$ (kept), $S_3 \approx 1.12$ (rejected), $S_4 \approx 0.64$ (kept). So $\mathcal{K} = \{\hat{\mathbf{p}}_2, \hat{\mathbf{p}}_4\}$ and $\hat{\mathbf{p}}_{\text{final}} = \frac{1}{2}[(1.5, -1.2) + (0.2, -1.5)] = (0.85, -1.35)$.

The clean solution S_1 gets thrown out, and two of the three contaminated ones survive and get averaged into a fix that's far off. This isn't a rare edge case – the median treats x and y independently and has no idea which subset contains the bad anchor, so it only rejects an outlier when the contaminated subsets are spread out and outnumbered by a tight clean cluster. This is the main failure mode discussed in §V-D.3, and it is why a full pairwise-distance clustering approach (see §V-G improvements) is worth considering instead of the simple median used here.

3) Assumptions, Advantages, Limitations

Advantages: when exactly one of four anchors is bad and the three unaffected subsets agree while the affected one diverges past τ, the median filter gives cheap robustness with no explicit robust loss or classifier needed.

Limitations: not a joint MLE over all four ranges – it trades statistical efficiency for a simple consensus scheme. The median is a heuristic, not a guaranteed-correct classifier, and can fail with two-or-more bad anchors, or even one, if the divergence pattern is

unfavorable. If fewer than three subsets converge, there is no median filtering at all – just an unfiltered mean, with no robustness guarantee.

Complexity: four independent LM solves ($\approx 4\times$ the cost of one three-anchor solve) plus $O(1)$ for the median/distance step.

E. Two-Anchor Solver: Closed-Form Bilateration with Motion Disambiguation

1) Motivation

With only two anchors, range alone doesn't pin down the position: two circles generally intersect at two points, and there is no third range to break the tie. This case has an exact closed form though (unlike $n \geq 3$), so instead of an iterative solve, the two candidates are found algebraically and picked between using the tag's recent motion (§V-E.4).

2) Ground-Plane Projection

Since anchors sit above the tag, each slant range ρ_i first needs converting to a horizontal range:

$$r_i' = \sqrt{\max(0,\ \rho_i^2 - (h - z_i)^2)}, \quad i \in \{1, 2\}. \tag{15}$$

The $\max(0, \cdot)$ guards against a small negative value under noise when the tag is nearly right below an anchor.

3) Circle Intersection via the Radical Axis

Let $\mathbf{a}_1, \mathbf{a}_2$ be the ground-plane anchor positions, $d = \|\mathbf{a}_2 - \mathbf{a}_1\|$ the baseline, r_1', r_2' the projected ranges. The tag lies on both circles:

$$\|\mathbf{p} - \mathbf{a}_1\|^2 = r_1'^2, \qquad \|\mathbf{p} - \mathbf{a}_2\|^2 = r_2'^2. \tag{16}$$

Subtracting cancels the quadratic term, leaving a linear equation – the radical axis. Projecting onto the baseline unit vector $\mathbf{u} = (\mathbf{a}_2 - \mathbf{a}_1)/d$ gives the signed distance from $\mathbf{a}_1$ to where the radical axis crosses the baseline:

$$a = \frac{r_1'^2 - r_2'^2 + d^2}{2d}. \tag{17}$$

The perpendicular offset follows from Pythagoras on the right triangle formed by $\mathbf{a}_1$, the foot point, and the intersection:

$$h' = \sqrt{\max(0,\ r_1'^2 - a^2)}. \tag{18}$$

With $\mathbf{v} = (-u_y, u_x)$ (perpendicular to $\mathbf{u}$), the two candidates are

$$\mathbf{p}_{\text{foot}} = \mathbf{a}_1 + a\mathbf{u}, \quad \mathbf{c}_1 = \mathbf{p}_{\text{foot}} + h'\mathbf{v}, \quad \mathbf{c}_2 = \mathbf{p}_{\text{foot}} - h'\mathbf{v}. \tag{19}$$

Both candidates satisfy (16) exactly.

4) Degenerate Cases

Disjoint circles ($d > r_1' + r_2'$): fall back to the point dividing the baseline by the range ratio,

$$\alpha = \frac{r_1'}{r_1' + r_2'}, \qquad \hat{\mathbf{p}} = \mathbf{a}_1 + \alpha(\mathbf{a}_2 - \mathbf{a}_1), \tag{20}$$

a reasonable compromise point, not a formal least-squares estimate.

Nested circles ($d + \min(r_1', r_2') < \max(r_1', r_2')$): pick the point on the larger circle closest to the disambiguation prior $\mathbf{p}_{\text{prior}}$,

$$\hat{\mathbf{p}} = \mathbf{c} + r\,\frac{\mathbf{p}_{\text{prior}} - \mathbf{c}}{\|\mathbf{p}_{\text{prior}} - \mathbf{c}\|}, \quad (\mathbf{c}, r) = \begin{cases} (\mathbf{a}_1, r_1') & r_1' \geq r_2' \\ (\mathbf{a}_2, r_2') & \text{otherwise} \end{cases}. \tag{21}$$

Proper intersection: pick whichever of $\mathbf{c}_1, \mathbf{c}_2$ is closer to $\mathbf{p}_{\text{prior}}$ (below).

5) Disambiguation via Motion Extrapolation

Two genuine circle intersections can't be told apart from a single instant of range data – this is a real structural ambiguity, not a noise artifact, so it needs outside information: here, the tag's recent trajectory. Using the last two accepted geometric-solver fixes (not the EKF's own state estimate – deliberately, so a drifting filter can't feed back into its own disambiguation), a constant-velocity extrapolation gives

$$\mathbf{p}_{\text{pred}} = 2\,\mathbf{p}_{\text{solver}}(t-1) - \mathbf{p}_{\text{solver}}(t-2). \tag{22}$$

This is only ever used as $\mathbf{p}_{\text{prior}}$ for tie-breaking – it is never fed into the Kalman filter as a measurement. Priority order for $\mathbf{p}_{\text{prior}}$: (1) the constant-velocity prediction (22) if recent history exists; (2) the last published position, if not too stale (`DISAMBIG_STALE_SEC`); (3) the anchor-field centroid, at cold start or after a long outage. Given $\mathbf{p}_{\text{prior}}$:

$$\hat{\mathbf{p}} = \begin{cases} \mathbf{c}_1 & \|\mathbf{c}_1 - \mathbf{p}_{\text{prior}}\| \leq \|\mathbf{c}_2 - \mathbf{p}_{\text{prior}}\| \\ \mathbf{c}_2 & \text{otherwise} \end{cases}. \tag{23}$$

6) Worked Example

Anchors $A_3 = (-4, 0)$, $A_2 = (4, 0)$, tag at true position $(1, 2, 0.5)$. Slant ranges: $\rho_3 = \sqrt{29.25}\,\text{m}$, $\rho_2 = \sqrt{13.25}\,\text{m}$. Ground ranges via (15): $r_1' = \sqrt{29} \approx 5.385\,\text{m}$, $r_2' = \sqrt{13} \approx 3.606\,\text{m}$.

Baseline $d = 8\,\text{m}$, $\mathbf{u} = (1, 0)$. From (17): $a = (29 - 13 + 64)/16 = 5.0\,\text{m}$. Foot point $\mathbf{p}_{\text{foot}} = (-4, 0) + 5(1, 0) = (1, 0)$, and $h' = \sqrt{29 - 25} = 2.0\,\text{m}$. With $\mathbf{v} = (0, 1)$: $\mathbf{c}_1 = (1, 2)$, $\mathbf{c}_2 = (1, -2)$. $\mathbf{c}_1$ matches the true position; $\mathbf{c}_2$ is the mirror-image ghost across the baseline.

With prior fixes $(0.8, 1.8)$ and $(0.6, 1.6)$: $\mathbf{p}_{\text{pred}} = 2(0.8, 1.8) - (0.6, 1.6) = (1.0, 2.0)$. Distance to $\mathbf{c}_1$ is 0, to $\mathbf{c}_2$ is 4.0, so (23) correctly picks $\hat{\mathbf{p}} = \mathbf{c}_1 = (1, 2)$.

7) Assumptions, Advantages, Limitations

Why it works: any real trajectory is locally smooth over one ranging cycle, so "which candidate continues the recent heading" is a strong, basically free disambiguator with no extra hardware.

Failure modes: cold start / long outage (falls back to weaker priors with no heading info – effectively a coin flip if both candidates are equidistant); sharp turns or reversals (constant-velocity assumption breaks, disambiguator can lock onto the wrong candidate until a downstream consistency check catches it); motion collinear with the anchor axis ($\mathbf{p}_{\text{pred}}$ can land near the baseline itself, making the two candidates near-equidistant); near-tangency or near-nested circles (small range errors swing h' and the candidates a lot).

Assumptions: non-coincident anchors ($d > 10^{-6}$); locally constant velocity between cycles; accurate h, z_i.

Summary: exact and $O(1)$ whenever the geometry isn't degenerate, and the motion-aware tie-break beats a naive nearest-last-point rule under sustained directional motion. The downside is that it's fundamentally underdetermined by range alone and leans entirely on the motion heuristic, which degrades in the cases above.

F. GDOP as a Real-Time Confidence Signal

1) Formulation

Anchor geometry directly controls how much range noise turns into position error – quantified by Geometric Dilution of Precision (GDOP). With $A \equiv \mathbf{J}(\hat{\mathbf{p}})$ (the Jacobian from (7), each row a unit line-of-sight vector from an anchor to the tag):

$$\text{GDOP} = \sqrt{\text{tr}[(A^\top A)^{-1}]}. \tag{24}$$

Collinear or clustered anchors make $A^\top A$ near-singular, blowing up GDOP; an orthogonal spread minimizes it.

2) Variance Inflation

GDOP is turned into a bounded scale factor:

$$r_{\text{scale}} = \text{clip}\left(\frac{\text{GDOP}}{\texttt{GDOP_WARN_THRESH}}, 1, \texttt{GDOP_R_SCALE_MAX}\right),$$
$$\sigma^2_{\text{pos}} = \texttt{UWB_BASE_SIGMA_SQ} \times r_{\text{scale}}. \tag{25}$$

This just scales the filter's trust in the fix based on live geometry – it doesn't touch the fix itself.

3) Worked Example

For $\hat{\mathbf{p}} = (1, 1)$ with the same three anchors as before, the unit line-of-sight matrix is

$$A = \begin{bmatrix} -0.3162 & 0.9487 \\ 0.9487 & -0.3162 \\ -0.9806 & -0.1961 \end{bmatrix}.$$

$\text{tr}[(A^\top A)^{-1}] \approx 1.6035$, so GDOP ≈ 1.267. With $\texttt{GDOP_WARN_THRESH} = 2.5$: $r_{\text{scale}} = \text{clip}(1.267/2.5, 1, 4.0) = 1.0$ – floored at 1, so σ^2_{pos} stays at baseline. Good, orthogonal geometry, no inflation needed.

G. The Complete Solver Pipeline

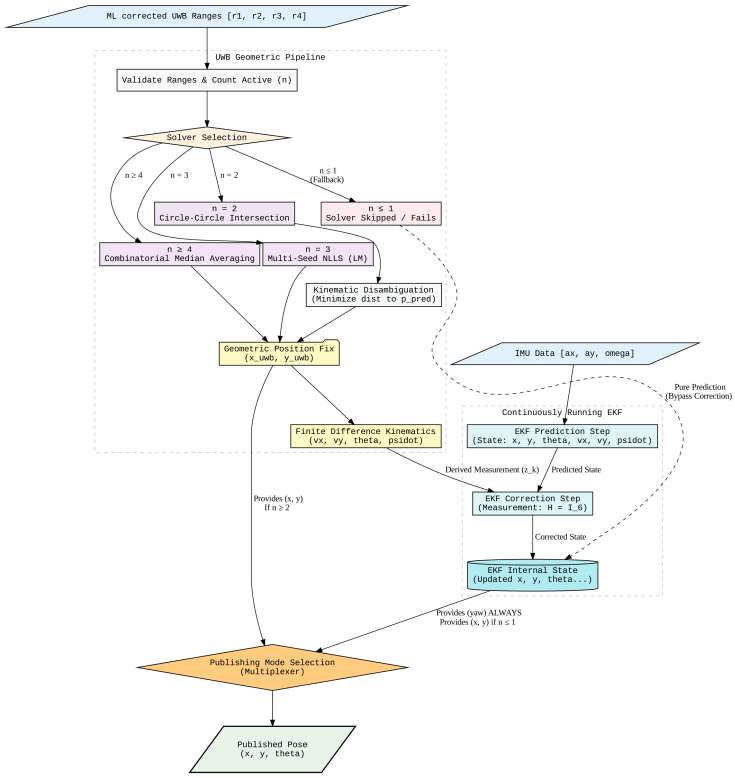


Figure 5: Localization Node v26: the geometric pipeline picks a solver based on active anchor count n, feeding the EKF a kinematic measurement. The EKF always publishes yaw, and takes over position entirely when $n \leq 1$.

H. Comparison of the Three Solvers

Table 4: Solver comparison summary

Solver	Technique	Anchors	Fails when
Three-anchor	Multi-start LM NLLS	3	Near-collinear anchors; residual above threshold for every seed
Four-anchor	4×3-anchor NLLS + median + mean	4	$<$ 3 subsets converge; empty candidate set; median rejects the clean fix
Two-anchor	Closed-form radical axis + motion tie-break	2	Disjoint/nested circles; near-tangency; cold start; sharp turns; motion along baseline

I. Discussion: Standard vs. Original Contributions

The NLLS cost, Jacobian, LM update, and radical-axis derivation are all standard, correctly-applied methods, consistent with established bilateration approaches. The numerical guards (`max(0,·)` clamps, baseline checks) just keep things well-behaved without changing the underlying math.

The multi-start grid and median filter are solid engineering choices, but the real original piece here is the two-anchor disambiguator: a stateless, two-point extrapolation kept fully decoupled from the fusion filter, specifically to avoid feedback loops if the filter drifts. That separation sets it apart from broader Bayesian motion-assisted approaches and is worth benchmarking against a naive last-known-point baseline.

Proposed improvements: (i) **Weighted NLLS** – use per-anchor variance σ_i^2 directly in the cost instead of inflating variance after the fact via GDOP; (ii) **Analytic Jacobian** – give LM the closed-form Jacobian instead of relying on finite differences, for better accuracy and speed; (iii) **Robust joint optimization** – replace the four-anchor subset-averaging with one joint weighted NLLS fit using a robust loss (Huber or Cauchy) to use all four ranges at once; (iv) **Covariance-aware disambiguation** – switch the two-anchor tie-break to a Mahalanobis distance against the filter's predicted covariance, but only when no UWB-only motion history is available, to keep the independence property intact.

VI. Machine-Learning UWB Distance Correction Model

A. Introduction and Scope

This section describes the machine learning model used to correct raw distance readings produced by the UWB ranging sensor. Raw distance measurements from UWB hardware are affected by signal conditions such as power level and signal quality, which introduce a predictable measurement offset. The model described here learns to estimate that offset and correct the raw distance so that the final reported distance is more accurate. Scope: explain how the model processes raw sensor data (data binning); explain the internal building blocks of the model (decision trees); explain how those building blocks are combined (boosting and bagging); document the exact production configuration, features, and performance metrics; document how the model's output should be interpreted and applied.

B. Model Overview

The final production model is summarized in Table V.

Table 5: UWB distance correction model summary

Property	Value
Model type	HistGradientBoostingRegressor (Scikit-learn)
Trees per sub-model	400
Maximum tree depth	5
Independent sub-models	3 (random seeds: 11, 23, 42)
Final prediction	Average of the 3 sub-model outputs
Task type	Regression – predicts a continuous distance correction
Output	A single floating-point value: the distance offset in millimetres (mm)

In plain terms: the model looks at 13 numbers describing a single UWB reading (raw distance, signal power, signal quality, etc.) and outputs one number: how many millimetres the raw distance reading should be corrected by.

The subsections below build up this model piece by piece: first how the raw data is prepared (binning), then how a single decision tree works, then how 400 trees are chained together (boosting), and finally how 3 independent models are combined (bagging).

C. Preparing the Data: Feature Binning

Before any tree is built, every sensor feature (for example `RX_Power`) is converted from a continuous, high-precision number into one of 256 discrete "bins" (buckets). This step is called binning or discretization, and it is what the word "Hist" in `HistGradientBoostingRegressor` refers to ("Histogram-based").

1) Worked Example

Consider raw `RX_Power` sensor readings, which can take an unlimited number of decimal values: $-89.3, -79.1, -75.4, -82.7, -68.5, -91.2$ dBm. The full range of possible `RX_Power` values is divided into 256 fixed-width buckets, for example: Bin 0: -100 to -95 dBm; Bin 1: -95 to -90 dBm; Bin 2: -90 to -85 dBm; Bin 3: -85 to -80 dBm; Bin 4: -80 to -75 dBm; Bin 5: -75 to -70 dBm; ... up to Bin 255: $+40$ to $+50$ dBm. Each raw reading is then replaced by the number of the bin it falls into.

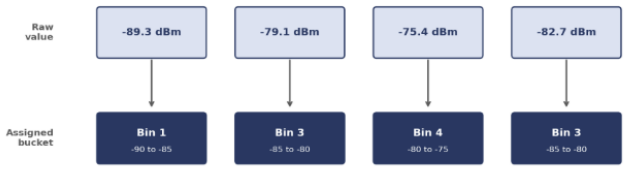


Figure 6: Raw sensor values are sorted into fixed bins.

The boundary values between bins $(-100, -95, -90, -85, \ldots, +40, +50)$ are called the bin edges. With 256 bins there are 255 edges.

2) Why Binning Is Used

Benefit 1 – Speed. Without binning, every comparison inside a tree is a slow floating-point check, e.g. "Is `RX_Power` > -79.1234567891234?". With binning, the same comparison becomes a simple integer check, e.g. "Is `bin_number` > 50?". Integer comparisons are dramatically faster, giving a 10–100× speed-up in both training and inference (prediction).

Benefit 2 – Memory Efficiency. Storing bin numbers (`uint8`, 1 byte) instead of raw float64 values (8 bytes) gives an 8× reduction in memory usage – 1 MB vs. 8 MB for 1 million samples.

Benefit 3 – Prevents Overfitting. Without binning, a tree could split on an extremely specific value (e.g. "`RX_Power` > -79.1234567"), which risks memorizing noise in the training data rather than learning a genuine pattern. With binning, a tree can only split on one of the fixed bin boundaries, which forces the model to generalize instead of memorizing individual data points.

Summary: every feature is divided into 256 bins. For a new reading, each of its features is first assigned a bin number, and every subsequent tree comparison works on these bin numbers (simple integers) rather than the original raw floating-point values.

D. Building Block: A Single Decision Tree

After binning, the model is built from decision trees. A decision tree makes a prediction by asking a sequence of yes/no questions about the input data, narrowing down to a final answer.

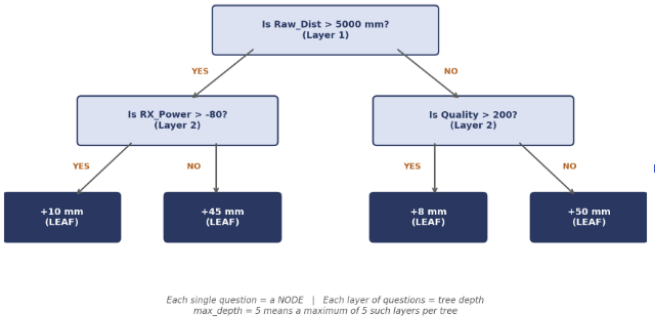


Figure 7: A single decision tree with `max_depth = 5` (simplified to 2 layers for illustration).

Key terminology: a **Node** is a single yes/no question, e.g. "Is `Raw_Dist` > 5000 mm?"; a **Layer** is all the questions at the same depth in the tree (asked in parallel across different branches); a **Leaf** is an end point of the tree that returns a final numeric answer (no further questions); **Max depth** is the maximum number of layers a tree is allowed to grow to.

1) Why Max Depth Matters

A shallow tree (depth 2) is simple and fast, but not very accurate – it cannot capture complex patterns. A medium tree (depth 5, used in this model) is more complex, more accurate, and still efficient to run. A very deep tree (depth 20+) can have millions of end points – too complex, it overfits by memorizing training data instead of generalizing. This model deliberately restricts every tree to `max_depth = 5` to prevent overfitting while retaining enough accuracy.

E. Combining Trees: Gradient Boosting

A single tree is not accurate enough on its own, so the model chains 400 trees together within each sub-model. This is called gradient boosting: each new tree is trained specifically to correct the mistakes made by the trees before it.



Figure 8: 400 trees chained sequentially, each correcting the previous trees' errors.

Tree 1 learns the overall pattern and makes some mistakes. Tree 2 looks specifically at Tree 1's mistakes and specializes in fixing them. Tree 3 looks at the mistakes still remaining and fixes those. This continues until Tree 400 fixes the last, smallest edge cases. Because each tree depends on the output of the trees before it, the 400 trees are connected, not independent – this set of 400 connected trees makes up one sub-model.

F. Combining Models: Bagging (Ensembling)

The 400-tree boosted structure described above forms one complete sub-model. This production model does not rely on a single sub-model – it independently trains 3 such sub-models (each with 400 trees, 1,200 trees in total) using three different random seeds, and then averages their outputs. This technique is called bagging (Bootstrap Aggregating).

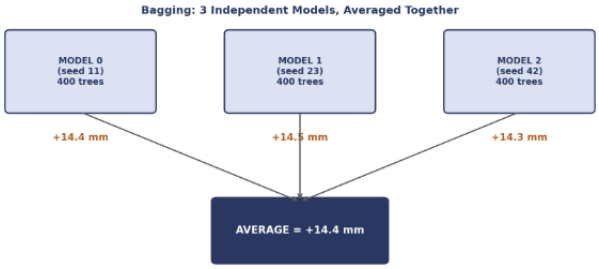


Figure 9: Three independent models are trained and their outputs averaged.

1) Worked Example

Sub-model	Seed	Example predicted correction
Model 0	11	+14.4 mm
Model 1	23	+14.5 mm
Model 2	42	+14.3 mm

Final averaged correction $= (+14.4 + 14.5 + 14.3)/3 = +14.4$ mm. $\texttt{Corrected_Distance} = \texttt{Raw_Distance} + 14.4$ mm.

2) Why 3 Models Instead of 1?

Each sub-model is trained with slightly different starting conditions (seeds 11, 23, and 42), so each one arrives at a slightly different estimate of the correction. For example, one sub-model might estimate the error at +12 mm, another at +10 mm, and another at +14 mm – averaging these to +12 mm produces a result that is more stable than trusting any single sub-model.

Note: a seed initializes the random number generator, ensuring identical training data is shuffled differently across models to create slight prediction variations that improve overall stability.

Why this works: bagging trains multiple models, lets each one "vote" with its own prediction, and takes the average. If any one model is having an off day on a particular reading, the other models balance it out, producing a steadier, more reliable final prediction.

G. Final Model Configuration

Table VI lists the exact hyperparameters used to train each of the 3 production sub-models.

Table 6: Production hyperparameters

Parameter	Value	Meaning
Loss function	absolute_error (MAE)	Optimized to minimize average absolute error in mm
Link function	Identity	Output used directly, no transformation
Learning rate	0.05	Controls how much each new tree is allowed to correct – small steps for stability
max_iter (trees)	400	All 400 trees used; early stopping never triggered
max_depth	5	Maximum number of layers per tree
max_leaf_nodes	31	Maximum number of end points per tree
min_samples_leaf	20	Minimum training examples required to form a leaf
l2_regularization	1.0	Penalizes overly complex trees to reduce overfitting
Bin threshold	256 bins (255 edges)	See §VI-C
Early stopping	Enabled (patience=10, tol=10^{-7})	Configured, but not triggered during training

1) Ensemble Totals

Sub-model	Seed	Trees	Total nodes
Model 0	11	400	22,510
Model 1	23	400	22,714
Model 2	42	400	22,650
Ensemble total	—	1,200	∼67,874

On average, each tree contains approximately 56 nodes and 29 leaves.

H. Input Features and Feature Engineering

The model uses 13 input features per reading. Four are collected directly from the UWB chip, and the remaining nine are calculated from those four using fixed formulas.

1) Directly Collected From the Sensor

#	Feature	Example	Description
1	Raw_Dist_mm	5000 mm	Raw distance reading
2	RX_Power	−80 dBm	Received signal strength
3	FP_Power	−82 dBm	First pulse signal strength
4	Delta	0.5 dBm	RX_Power − FP_Power
13	Quality	240 (0–255)	Signal quality score, from chip

2) Calculated (Engineered) Features

These are derived from the four collected values above, using example values $\texttt{Raw_Dist_mm} = 5000$, $\texttt{RX_Power} = -80$, $\texttt{FP_Power} = -82$, $\texttt{Delta} = 0.5$:

#	Feature	Formula
5	Pwr_Diff	RX_Power − FP_Power
6	Raw_Delta	Raw_Dist_mm × Delta
7	Power_Delta	RX_Power × Delta
8	Raw_Rx	Raw_Dist_mm × RX_Power
9	Rx_Fp_Prod	RX_Power × FP_Power
10	Raw_Sq	Raw_Dist_mm^2
11	Phase_Sin	$\sin(2\pi \cdot (\texttt{Raw_Dist_mm} \bmod 46.2)/46.2)$
12	Phase_Cos	$\cos(2\pi \cdot (\texttt{Raw_Dist_mm} \bmod 46.2)/46.2)$

3) Feature Importance

Feature importance measures how often each feature was actually used by the trees to make a split decision. A higher percentage means the feature contributes more to the model's predictions.

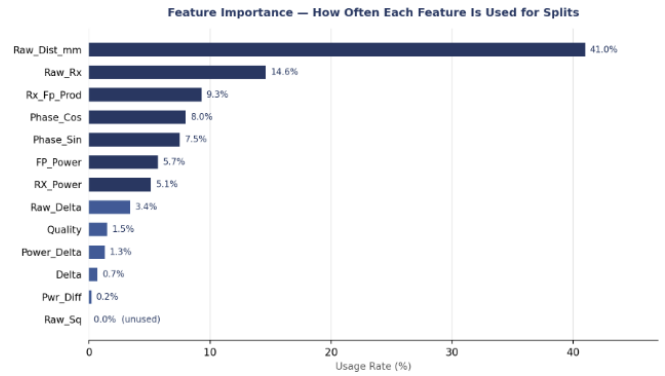


Figure 10: Usage rate of each feature across all trees in the ensemble.

Rank	Feature	Usage rate
1	Raw_Dist_mm	41.0%
2	Raw_Rx	14.6%
3	Rx_Fp_Prod	9.3%
4	Phase_Cos	8.0%
5	Phase_Sin	7.5%
6	FP_Power	5.7%
7	RX_Power	5.1%
8	Raw_Delta	3.4%
9	Quality	1.5%
10	Power_Delta	1.3%
11	Delta	0.7%
12	Pwr_Diff	0.2%
13	Raw_Sq	0.0% (unused)

Observation: `Raw_Dist_mm` alone accounts for 41% of all split decisions, confirming it is by far the strongest predictor. `Raw_Sq` (the squared distance) was never used by any tree and is a candidate for removal in a future iteration.

I. Model Output and Prediction Workflow

The model outputs a single number: a distance correction offset in millimetres.

Output sign	Meaning	Action
Positive	Raw distance OVER-reported	Subtract correction from raw distance
Negative	Raw distance UNDER-reported	Add |correction| to raw distance
Zero (rare)	No correction needed	Use raw distance as-is

1) End-to-End Prediction Steps

1. Collect the 4 raw values from the UWB chip: Raw_Dist_mm, RX_Power, FP_Power, Delta.
2. Calculate the 9 engineered features using the formulas of §VI-H.2.
3. Assign each of the 13 features to its corresponding bin (§VI-C).
4. Pass the binned features through each of the 3 sub-models (each running its 400 chained trees, §VI-D–VI-E).
5. Average the 3 sub-model outputs to get the final correction (§VI-F).
6. Apply the correction to the raw distance as shown above.

J. Training Data and Performance Metrics

Metric	Value
Training data source	61 UWB sessions
Distance range covered	3.55 mm – 10,242 mm
Number of input features	13
Out-of-fold overall RMSE	48.10 mm

1) Per-Model Training Scores

Sub-model	Seed	Baseline pred.	Final train	Final val.
Model 0	11	78.93	−9.603	−10.163
Model 1	23	78.575	−9.750	−10.146
Model 2	42	78.75	−9.735	−10.181

Note: train/validation scores are reported as negative MAE (Scikit-learn convention) – values closer to zero indicate lower error.

K. Appendix: Raw Model Metadata

The following values are the raw metadata exported directly from the trained model objects, provided here for engineering reference and reproducibility.

Table 7: model_0

Field	Value
baseline_prediction	78.93000000000029
n_iter_	400
n_trees_per_iteration_	1
n_features_in_	13
random_state	11
final_train_score	−9.60289824109379
final_validation_score	−10.162765776173256
bin_mapper_n_bins	256
bin_mapper_subsample	200000
total_trees	400
total_nodes	22510

Table 8: model_1

Field	Value
baseline_prediction	78.57499999999982
n_iter_	400
n_trees_per_iteration_	1
n_features_in_	13
random_state	23
final_train_score	−9.750091807653506
final_validation_score	−10.145742136391231
bin_mapper_n_bins	256
bin_mapper_subsample	200000
total_trees	400
total_nodes	22714

Table 9: model_2

Field	Value
baseline_prediction	78.75
n_iter_	400
n_trees_per_iteration_	1
n_features_in_	13
random_state	42
final_train_score	−9.73488501670009
final_validation_score	−10.181411655099616
bin_mapper_n_bins	256
bin_mapper_subsample	200000
total_trees	400
total_nodes	22650

VII. ONNX Model Conversion for Real-Time Deployment

A. The Problem: RPi 5 Was Crashing at 4 Anchors

1) How the UWB System Works

The UWB tag cycles through active anchors and outputs readings over serial at increasing rates as more anchors are added (see Table III). Each anchor reading must be corrected by the ML model once per reading, to calculate the distance error.

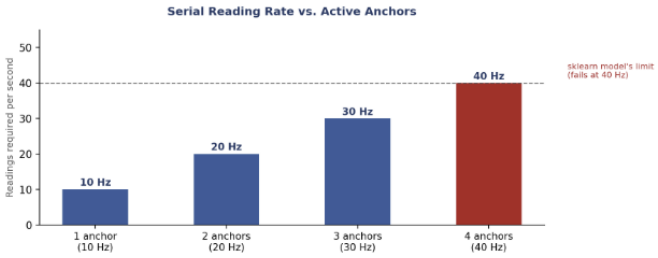


Figure 11: Required reading rate increases with each additional anchor.

2) What Happened

With the original scikit-learn model (`.pkl` file): at 1–3 anchors (10–30 Hz), the system worked fine – the model was slow, but fast enough to keep up. At 4 anchors (40 Hz), it failed: the model couldn't process readings fast enough, and new readings arrived faster than the model could finish correcting the previous ones. When this happens, the following chain of failures occurs: unprocessed readings pile up in the serial buffer → the buffer fills and overflows → data gets corrupted or lost → the ROS node crashes → the entire UWB positioning system stops working.

Key point: this is a speed problem, not an accuracy problem. The model wasn't wrong – it was just too slow.

3) Why Just Optimizing Python Didn't Work

The scikit-learn library runs in Python, which adds overhead to every single prediction call: object creation and deletion, type checking, memory allocation, and Python interpreter overhead. With the model running 40 times per second, this overhead adds up quickly. The fix needed to remove this overhead entirely, not just tune it.

B. The Solution: ONNX

1) What Is ONNX?

ONNX (Open Neural Network Exchange) is a file format that stores trained models in a way that any application can run, without needing the original ML library.

In plain terms: the original model (`.pkl`) is a recipe written by scikit-learn, in scikit-learn's format, that only scikit-learn can read. An ONNX model is the same recipe translated into a universal format that any compatible runtime can execute.

2) Why ONNX Is Faster

The original model was stored as Python objects. Every time a prediction was needed, Python had to unpack the Python objects, check data types, allocate memory, and do the calculation. ONNX removes all of this – the model is pre-compiled into a structure that a fast C++ engine can execute directly, with no Python overhead.

3) Why ONNX Is Better for Raspberry Pi

The Raspberry Pi has limited CPU power and RAM. The original approach wasted both: (i) *Storage* – Python libraries (scikit-learn, numpy, scipy) are large; ONNX only needs one small runtime library. (ii) *Speed* – each prediction call went through Python, which is slow; ONNX predictions run in compiled C++ code, which is much faster. (iii) *CPU load* – less CPU usage per prediction means less heat generated.



Figure 12: Software stack before and after ONNX conversion.

C. What Changed in the Model

1) Before vs. After

Property	Before (.pkl)	After (ONNX)
Format	Python pickle (.pkl)	ONNX graphs inside a pickle wrapper
Contents	3 scikit-learn HistGradientBoostingRegressor objects	3 compiled ONNX TreeEnsembleRegressor operators
Each object/operator	400 decision trees stored as Python objects	All 400 trees compiled into one fast operator
Requires	scikit-learn installed and loaded	ONNX Runtime only (not scikit-learn)

2) File Details

The converted model (`uwb_precision_model_onnx_v6.pkl`) contains:

Item	Details
Number of models	3 (same as before)
Trees per model	400 (same as before)
ONNX format	IR version 8, opset 15
Data type	Float32 (32-bit floating point)
File size per model	∼830 KB
Total file size	∼2.5 MB
Inference speed	3 models together: 0.035 ms per prediction

Most important change: all 400 trees per model are now compiled into a single fast operator, not stored as 400 separate Python objects.

D. How to Use the ONNX Model

1) Installation

On the Raspberry Pi, install only:

```
pip install onnxruntime
```

No need to install scikit-learn, numpy, scipy, or other heavy libraries.

2) Code Example

```
import pickle
import onnxruntime as rt
import numpy as np

# Load the model
with open('uwb_precision_model_onnx_v6.pkl', 'rb') as
    f:
   pkg = pickle.load(f)

# Create ONNX runtime sessions
sessions = [
   rt.InferenceSession(onnx_bytes,
      providers=['CPUExecutionProvider'])
   for onnx_bytes in pkg['models']
]

# Make a prediction
def correct_distance(features):
   # features: list of 13 values in this order:
   # [Raw_Dist_mm, RX_Power, FP_Power, Delta,
   # Pwr_Diff, Raw_Delta, Power_Delta, Raw_Rx,
   # Rx_Fp_Prod, Raw_Sq, Phase_Sin, Phase_Cos,
   # Quality]
   x = np.array([features], dtype=np.float32)
   # Run all 3 models
   outputs = []
   for sess in sessions:
      result = sess.run(None, {'input': x})
      outputs.append(result[0][0][0])
   # Average the results
   correction = sum(outputs) / len(outputs)
   return correction
```

3) Important Notes

- The input tensor name is `input` (not `float_input`).
- All 13 features must be in the exact order shown above.
- The model outputs a single float value: the distance correction.
- No scikit-learn needed at runtime – only when originally converting to ONNX.

E. Performance Improvement

1) Speed Comparison

Metric	scikit-learn	ONNX
Per-prediction time	High (varies)	0.035 ms (all 3 models)
At 40 Hz load	Can't keep up, buffer overflows	Runs 28,000 predictions/sec max (600× overhead)
Thermal load	Higher	Lower
CPU usage	High per prediction	Low per prediction

2) Why This Fixes the 4-Anchor Problem

Before: the model couldn't process 40 readings/sec → readings piled up → buffer overflow. After: the model can process 28,000 readings/sec → handles 40 readings/sec easily with huge margin.

F. Accuracy Impact

The original scikit-learn model uses 64-bit floating point (double precision). The ONNX model uses 32-bit floating point (single

precision). The difference: less than 0.0000001 mm per prediction.

This is completely irrelevant because: UWB sensors have real-world error margins of ±50 mm; training data noise is much larger than 0.0000001 mm; the difference doesn't matter for any practical use.

G. Summary

Aspect	Before (scikit-learn)	After (ONNX)
Works at 3 anchors (30 Hz)	Yes, barely	Yes, comfortably
Works at 4 anchors (40 Hz)	No – crashes	Yes – huge margin
Model accuracy	Baseline	Same (0.0000001 mm difference)
File size	4.9 MB	2.5 MB
Library requirements	scikit-learn + numpy + scipy	ONNX Runtime only
Per-prediction time	Variable, slow	Consistent, fast
Installation on RPi	Complex, large footprint	Simple: `pip install onnxruntime`

Bottom line: the ONNX model solves the fundamental problem of inference speed. It uses the exact same trained trees but executes them 10–100× faster through a compiled runtime, making it practical to run at 40 Hz without buffer overflow or system crashes.

VIII. A Five-State Extended Kalman Filter for IMU/UWB Fusion

This section derives, from first principles, a five-state Extended Kalman Filter (EKF) for localizing the wheeled ground vehicle using the IMU for prediction and the UWB ranging system (via the geometric solvers of Section V) for correction. State-space models provide a uniform first-order representation of a dynamical system, $\dot{\mathbf{x}}(t) = A\mathbf{x}(t)+B\mathbf{u}(t), \mathbf{y}(t) = C\mathbf{x}(t)+D\mathbf{u}(t)$, which is exact only for linear time-invariant (LTI) systems; a nonlinear system $\dot{\mathbf{x}} = f_c(\mathbf{x}, \mathbf{u})$ must first be linearized about an operating point before the linear machinery of closed-form propagation becomes available, following the general linearization procedure $A = \partial f/\partial \mathbf{x}|_{(\mathbf{x}_e,\mathbf{u}_e)}$.

This section applies exactly that linearization procedure to tracking the planar pose and velocity of a wheeled ground vehicle by fusing an IMU (used for prediction) with a UWB ranging system (used for correction), in the single-stage, fully observable EKF architecture shown in Fig. 14. The filter's continuous-time kinematics are nonlinear only through a single heading-dependent rotation block, which makes it an unusually transparent worked example of Jacobian linearization: every step of §VIII-D below is a direct instance of the general recipe $A \triangleq \partial f/\partial \mathbf{x}|_{\hat{\mathbf{x}}}$, $B \triangleq \partial f/\partial \mathbf{u}|_{\hat{\mathbf{x}}}$.

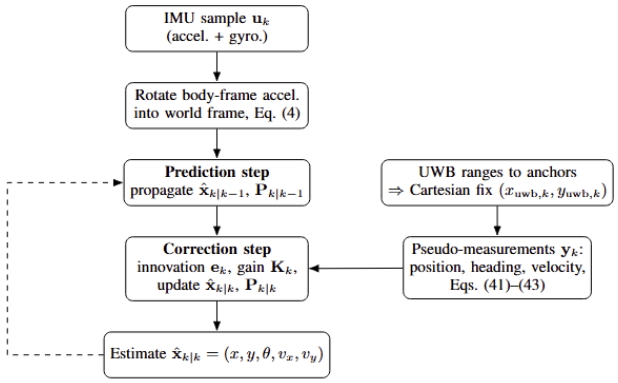


Figure 13: Single-stage EKF localization pipeline. The IMU drives the prediction step (§VIII-E); the UWB system drives the correction step (§VIII-F–VIII-G). Both steps act on one joint five-state estimate and one joint covariance matrix.

Every symbol used in this section is collected in Table X, at the end of the section, ahead of Table XI, so that a reader can look up any symbol without searching the body text. The convention follows standard state-space practice together with the frame conventions of §VIII-B: a quantity with no superscript is expressed in the fixed world frame; a superscript (b) marks a body-frame quantity; a caret $\hat{}$ marks a filter estimate; and $\delta(\cdot)$ marks the (unknown) error between a true quantity and its estimate. Every equation that follows depends on the assumptions collected in Table XI.

A. State, Control, and Noise Vectors

A wheeled ground vehicle moving in a horizontal plane has three degrees of positioning freedom: two translational and one rotational. Predicting where the vehicle will be an instant later also requires its current rate of change of position and heading, so the state is augmented with velocity rather than limited to pose alone. The five-dimensional state vector is

$$\mathbf{x}_k = [x_k \;\; y_k \;\; \theta_k \;\; v_{x,k} \;\; v_{y,k}]^\top \in \mathbb{R}^5, \tag{1}$$

where (x, y) is the vehicle's world-frame position and θ is its yaw, measured counter-clockwise from the world x-axis, and (v_x, v_y) is its world-frame linear velocity. Every quantity in this section that carries no frame superscript is, by convention, a world-frame quantity; only body-frame quantities carry the superscript (b).

The vehicle's short-term motion is predicted from its IMU, which reports the body-frame accelerations and the turning rate at every sample:

$$\mathbf{u}_k = [a_x^{(b)} \;\; a_y^{(b)} \;\; \omega_{\text{imu}}]^\top \in \mathbb{R}^3. \tag{2}$$

Because these accelerometer readings are physically measured by a sensor bolted to the vehicle chassis, they are reported in the body frame, not the fixed world frame above.

No sensor is perfect and no kinematic model captures every physical effect (wheel slip, unmodeled vibration, and so on). This section does not carry an explicit additive process-noise term in the

continuous-time state equation; instead, the resulting uncertainty is accounted for entirely through the process noise covariance Q_k used in the covariance-propagation step, constructed explicitly in §VIII-E.3.

B. Body Frame versus World Frame

Two distinct coordinate frames appear in this derivation, and confusing them is the single most common source of implementation error, so the distinction is made concrete before any dynamics are written down.

The world frame is a single, fixed frame, chosen once and never rotating; it is convenient to align it with the UWB anchor constellation. All world-frame quantities carry no superscript. The body frame is attached to the vehicle chassis and rotates with it: its x-axis always points in the vehicle's current forward direction and its y-axis always points to the vehicle's current left, regardless of the vehicle's heading in the world frame. The accelerometer is bolted to the chassis and can only report accelerations along these body-fixed axes; this is why the control input above carries the (b) superscript.

Fig. 15 illustrates the two frames together with a single acceleration vector resolved into each. The body axes $x^{(b)}, y^{(b)}$ are rotated by the heading θ relative to the fixed world axes; the same physical acceleration has components $a_x^{(b)}, a_y^{(b)}$ along the body axes and a_x, a_y along the world axes, and the figure makes visually explicit how one set of components maps to the other under a rotation by θ – precisely the relationship formalized below.

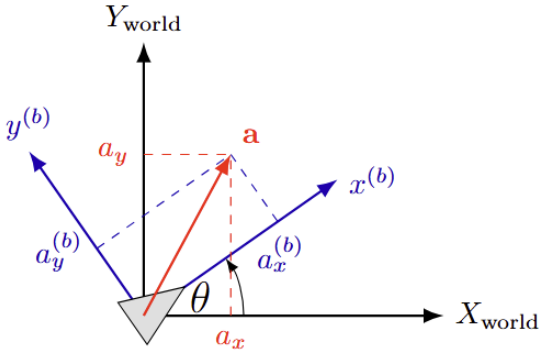


Figure 14: Body frame versus world frame. The vehicle's body-fixed axes $x^{(b)}, y^{(b)}$ (blue) are rotated by the heading θ relative to the fixed world axes $X_{\text{world}}, Y_{\text{world}}$. The same physical acceleration vector $\mathbf{a}$ has components $a_x^{(b)}, a_y^{(b)}$ along the body axes and a_x, a_y along the world axes; rotating the body components by θ produces the world components.

Rotating a vector from a frame tilted by θ into a frame tilted by 0 is accomplished by the standard planar rotation matrix,

$$\boxed{R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}}, \tag{3}$$

so that the world-frame acceleration is $\mathbf{a} = R(\theta)\mathbf{a}^{(b)}$, i.e., component-wise,

$$a_x = a_x^{(b)}\cos\theta - a_y^{(b)}\sin\theta, \qquad a_y = a_x^{(b)}\sin\theta + a_y^{(b)}\cos\theta. \tag{4}$$

This single rotation is the origin of every trigonometric term in this section, and, as shown in §VIII-D, it is the only source of nonlinearity in the entire prediction model.

C. Continuous-Time Nonlinear Kinematics

[Continuous-time kinematics] Under Assumptions A2–A4 of Table XI, the nonlinear continuous-time state equation $\dot{\mathbf{x}} = f_c(\mathbf{x}, \mathbf{u})$ is, in compact block-matrix form,

$$\dot{\mathbf{x}} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & \cos\theta & -\sin\theta \\ 0 & 0 & 0 & \sin\theta & \cos\theta \end{bmatrix}}_{\triangleq\, \text{blkdiag}(I_2, 1, R(\theta))} \begin{bmatrix} v_x \\ v_y \\ \omega_{\text{imu}} \\ a_x^{(b)} \\ a_y^{(b)} \end{bmatrix} \tag{5}$$

where I_2 is the 2×2 identity block acting on (v_x, v_y), the central 1 passes the gyroscope rate directly to $\dot{\theta}$, and $R(\theta)$ acts on the body-frame acceleration. Written out row by row, this is equivalent to the five scalar equations

$$\dot{x} = v_x, \qquad \dot{y} = v_y, \tag{6}$$

$$\dot{\theta} = \omega_{\text{imu}}, \tag{7}$$

$$\dot{v}_x = a_x^{(b)}\cos\theta - a_y^{(b)}\sin\theta \triangleq a_x, \tag{8}$$

$$\dot{v}_y = a_x^{(b)}\sin\theta + a_y^{(b)}\cos\theta \triangleq a_y. \tag{9}$$

Proof: the position rows are the definition of velocity, $v_x = \dot{x}$, $v_y = \dot{y}$, exactly, with no approximation. The heading row substitutes the gyroscope's measured turning rate directly for the true yaw rate under A4: over one short interval, the yaw rate is treated as the piecewise-constant value most recently reported by the gyroscope, rather than as a state governed by its own dynamics; this is why ω_{imu} appears in the input rather than as a sixth state. The velocity rows follow from §VIII-B: world-frame acceleration is body-frame acceleration rotated by $R(\theta)$, and, under A4, this rotated acceleration is the rate of change of world-frame velocity, $\dot{\mathbf{v}} = \mathbf{a}$. Collecting all five rows into one matrix equation is nothing more than stacking these two independent physical facts – "velocity and yaw rate integrate directly" and "acceleration must be rotated first."

Why this system is nonlinear: a linear system requires $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ for constant matrices A, B that do not themselves depend on the state. Four of the five rows above are already exactly this form: the I_2 block and the central 1 are fixed numbers, independent of x, y, θ, v_x, v_y. The obstruction is entirely contained in the $R(\theta)$ block: the heading θ – itself one of the five state variables – appears inside the transcendental functions $\sin\theta, \cos\theta$

that populate it. No constant matrix can reproduce this dependence exactly, so $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ cannot hold for any fixed A, B obtained by freezing $\mathrm{blkdiag}(I_2, 1, R(\theta))$ at one value of θ.

This matters because the Kalman filter's covariance recursion is a closed-form result valid only for linear systems; it cannot be applied directly here. This motivates the linearization of §VIII-D: not to replace the nonlinear model everywhere (the exact nonlinear model is still used for the state prediction itself, §VIII-E), but specifically to obtain the constant-at-an-instant matrix needed to propagate the covariance.

D. Jacobian Linearization

This subsection is the technical heart of the derivation. Four questions are answered in order: (1) what nonlinear model is being linearized – the two nonlinear rows for $\dot{v}_x, \dot{v}_y$ of §VIII-C; (2) around which point – the filter's own current estimate $\hat{\mathbf{x}}$, never the unknown true state, per A12; (3) which quantities end up linear – the error $\delta\mathbf{x} = \mathbf{x} - \hat{\mathbf{x}}$, not the state itself; (4) why $\sin\theta, \cos\theta$ still appear after linearization – answered in §VIII-D.3: no small-angle approximation is used anywhere; the trigonometric functions are evaluated numerically at the known current estimate $\hat{\theta}$, and it is only the unknown error $\delta\theta$ that is treated linearly.

1) The Linearization Principle

The filter never has access to the true state $\mathbf{x}$; it only has an estimate $\hat{\mathbf{x}}$. Write the unknown true state and input as the known estimate plus an unknown perturbation, $\mathbf{x} = \hat{\mathbf{x}} + \delta\mathbf{x}$, $\mathbf{u} = \hat{\mathbf{u}} + \delta\mathbf{u}$. Substituting into $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ and expanding f in a first-order Taylor series about $(\hat{\mathbf{x}}, \hat{\mathbf{u}})$,

$$\dot{\hat{\mathbf{x}}} + \dot{\delta\mathbf{x}} \approx f(\hat{\mathbf{x}}, \hat{\mathbf{u}}) + \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}},\hat{\mathbf{u}}} \delta\mathbf{x} + \left.\frac{\partial f}{\partial \mathbf{u}}\right|_{\hat{\mathbf{x}},\hat{\mathbf{u}}} \delta\mathbf{u} + O(\|\delta\mathbf{x}, \delta\mathbf{u}\|^2), \tag{10}$$

where the neglected terms are quadratic and higher in the perturbations, and are small whenever $\delta\mathbf{x}, \delta\mathbf{u}$ are small relative to the operating point – the assumption that justifies linearization, in the same sense that a small enough patch of a curved surface looks flat. The nominal trajectory is, by definition, itself governed by the exact nonlinear equation at the operating point,

$$\dot{\hat{\mathbf{x}}} = f(\hat{\mathbf{x}}, \hat{\mathbf{u}}), \tag{11}$$

which is precisely the prediction equation of §VIII-E. Subtracting this from the Taylor expansion cancels the zeroth-order term exactly, and dropping the higher-order remainder leaves the linear error-state equation

$$\dot{\delta\mathbf{x}} = A\,\delta\mathbf{x} + B\,\delta\mathbf{u}, \qquad A \triangleq \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}}, \quad B \triangleq \left.\frac{\partial f}{\partial \mathbf{u}}\right|_{\hat{\mathbf{x}}}. \tag{12}$$

Each entry of the Jacobian A is a partial derivative $\partial f_i / \partial x_j$, quantifying the sensitivity of the i-th equation of motion to the j-th state variable alone, holding every other state variable fixed.

Because $\hat{\mathbf{x}}, \hat{\mathbf{u}}$ are known numbers at the instant the Jacobian is evaluated, the entries of A and B – however complicated their algebraic form – are ordinary constants from the point of view of this equation. It is genuinely linear in the unknowns $\delta\mathbf{x}, \delta\mathbf{u}$: no products of unknowns, no squares of unknowns, no transcendental functions applied to unknowns. Because $\hat{\mathbf{x}}$ changes at every instant, A and B must be recomputed at every instant; the error system is therefore linear time-varying (LTV) rather than LTI: an LTI system has one fixed A for all time, which is impossible here given the $\sin\theta, \cos\theta$ terms, whereas an LTV system is a new, genuinely linear system built fresh at every step using the current estimate as a fixed numerical input to that step's Jacobian.

2) Worked Example: the $\dot{v}_x$ Row

Consider $\dot{v}_x = a_x^{(b)} \cos\theta - a_y^{(b)} \sin\theta$. Write the true heading as the known estimate plus an unknown error, $\theta = \hat{\theta} + \delta\theta$, and hold the input at its nominal value ($\delta\mathbf{u} = 0$) for clarity.

Step 1 – Taylor-expand the trigonometric functions. This is the only approximation used anywhere in this section. Because $\delta\theta$ is assumed small, $\cos$ and $\sin$ are each expanded to first order about $\hat{\theta}$:

$$\cos(\hat{\theta} + \delta\theta) \approx \cos\hat{\theta} - \sin\hat{\theta}\,\delta\theta, \quad \sin(\hat{\theta} + \delta\theta) \approx \sin\hat{\theta} + \cos\hat{\theta}\,\delta\theta. \tag{13}$$

This expansion does not replace $\sin\hat{\theta}$ or $\cos\hat{\theta}$ with anything simpler – $\hat{\theta}$ is a known estimate, not a small quantity, and no small-angle approximation is applied to it. What is expanded is the dependence on the unknown increment $\delta\theta$, which is assumed small.

Step 2 – Substitute into the dynamics. Substituting into the $\dot{v}_x$ equation,

$$\begin{aligned}\dot{v}_x &\approx a_x^{(b)}\big(\cos\hat{\theta} - \sin\hat{\theta}\,\delta\theta\big) - a_y^{(b)}\big(\sin\hat{\theta} + \cos\hat{\theta}\,\delta\theta\big) \\ &= \underbrace{\Big(a_x^{(b)}\cos\hat{\theta} - a_y^{(b)}\sin\hat{\theta}\Big)}_{=\dot{\hat{v}}_x\ \text{(nominal)}} - \underbrace{\Big(a_x^{(b)}\sin\hat{\theta} + a_y^{(b)}\cos\hat{\theta}\Big)}_{=a_y\ \text{(world-frame accel., at }\hat{\theta}\text{)}}\delta\theta.\end{aligned} \tag{14}$$

Step 3 – Identify and cancel the nominal term. The first bracketed term above is exactly the right-hand side of the $\dot{v}_x$ equation evaluated at $\hat{\theta}$ – the nominal prediction $\dot{\hat{v}}_x$, a known number computed once per cycle as part of the ordinary prediction step.

Step 4 – Subtract the nominal term. Subtracting $\dot{\hat{v}}_x$ from both sides, and discarding the already-negligible second-order term in $\delta\theta$,

$$\delta\dot{v}_x = \dot{v}_x - \dot{\hat{v}}_x \approx -a_y\,\delta\theta. \tag{15}$$

This is the entry that reappears formally as the $(4, 3)$ entry of A below. It says that the rate of change of the velocity error is proportional to the heading error $\delta\theta$, with proportionality constant $-a_y$. Here a_y is a number, computed from the current IMU sample and the current heading estimate $\hat{\theta}$ – it involves $\sin\hat{\theta}, \cos\hat{\theta}$ algebraically, but these are evaluated at a known angle, so a_y itself is just a scalar, refreshed once per cycle; and $\delta\theta$ is the unknown being reasoned about, appearing to the first power only – not inside a sine, not inside a cosine, not squared. This is precisely

what makes the equation linear in the unknown $\delta\theta$, notwithstanding that the coefficient multiplying it was derived from trigonometric functions: linearity is a statement about how the equation depends on the unknown perturbation, not a statement that no trigonometric function may appear anywhere in the coefficients.

3) *Why* $\sin\theta$ *and* $\cos\theta$ *Still Appear After Linearization*

It is easy to find it confusing that the linearized matrices, given in §VIII-D.4–VIII-D.5 below, still contain $\sin\theta$ and $\cos\theta$. The resolution is that no small-angle approximation is used anywhere in this section. It is never assumed that θ itself is small, and it is never approximated as $\sin\theta \approx \theta$, $\cos\theta \approx 1$; such an approximation would be physically wrong, since the vehicle's heading ranges over the full circle $(-\pi, \pi]$ as it drives around. What is assumed small is only the error $\delta\theta$ between the filter's current estimate $\hat{\theta}$ and the unknown true heading θ – reasonable because the filter is continuously corrected by UWB fixes (§VIII-G) and therefore never allowed to drift far from the truth. The worked example above makes the mechanism explicit: the Taylor expansion of Step 1 is applied to $\cos(\hat{\theta}+\delta\theta)$ and $\sin(\hat{\theta}+\delta\theta)$ as functions of the small quantity $\delta\theta$, while treating $\hat{\theta}$ itself as an arbitrary known constant – exactly as one Taylor-expands any smooth function $g(\hat{\theta}+\delta\theta) \approx g(\hat{\theta}) + g'(\hat{\theta})\delta\theta$ about a general point $\hat{\theta}$. The result retains $\sin\hat{\theta}, \cos\hat{\theta}$ exactly, evaluated at whatever the current heading estimate happens to be. These survive into the final Jacobian entries because they are coefficients multiplying the linear unknown $\delta\theta$, not because any approximation was skipped.

In summary: the EKF linearizes the dependence of the dynamics on the state estimation error, using Jacobians built from ordinary partial derivatives, while fully preserving the nonlinear trigonometric functions of the heading estimate both in the state-prediction equations (§VIII-E, which uses the exact nonlinear model) and in the numerical value of the Jacobian entries themselves. This is also why A and B must be recomputed at every cycle, and why the resulting linear system is LTV rather than LTI.

4) *The State Jacobian* A

Having built one entry of A by hand above, the remaining entries follow the same procedure, generalized: differentiate each right-hand side of §VIII-C with respect to each state variable, evaluate at the current estimate $\hat{\mathbf{x}}$, and place the result in a template matrix whose rows and columns are both ordered (x, y, θ, v_x, v_y).

[Continuous Jacobian A] Let $A(t) \triangleq \partial f_c/\partial \mathbf{x}|_{\hat{\mathbf{x}}(t)}$. Writing out every partial-derivative entry individually before evaluating any of them, the template is

$$A = \begin{bmatrix} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial y} & \frac{\partial \dot{x}}{\partial \theta} & \frac{\partial \dot{x}}{\partial v_x} & \frac{\partial \dot{x}}{\partial v_y} \\ \frac{\partial \dot{y}}{\partial x} & \frac{\partial \dot{y}}{\partial y} & \frac{\partial \dot{y}}{\partial \theta} & \frac{\partial \dot{y}}{\partial v_x} & \frac{\partial \dot{y}}{\partial v_y} \\ \frac{\partial \dot{\theta}}{\partial x} & \frac{\partial \dot{\theta}}{\partial y} & \frac{\partial \dot{\theta}}{\partial \theta} & \frac{\partial \dot{\theta}}{\partial v_x} & \frac{\partial \dot{\theta}}{\partial v_y} \\ \frac{\partial \dot{v}_x}{\partial x} & \frac{\partial \dot{v}_x}{\partial y} & \frac{\partial \dot{v}_x}{\partial \theta} & \frac{\partial \dot{v}_x}{\partial v_x} & \frac{\partial \dot{v}_x}{\partial v_y} \\ \frac{\partial \dot{v}_y}{\partial x} & \frac{\partial \dot{v}_y}{\partial y} & \frac{\partial \dot{v}_y}{\partial \theta} & \frac{\partial \dot{v}_y}{\partial v_x} & \frac{\partial \dot{v}_y}{\partial v_y} \end{bmatrix}. \tag{16}$$

Evaluating each of these twenty-five entries (proof below) and substituting, with a_x, a_y as in §VIII-C, gives the State Transition (Process) Jacobian,

$$\boxed{A = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -a_y & 0 & 0 \\ 0 & 0 & a_x & 0 & 0 \end{bmatrix}}. \tag{17}$$

Proof: each row of A is obtained by differentiating the corresponding row of §VIII-C with respect to each of the five state variables in turn, holding all others fixed.

Row 1 ($\dot{x} = v_x$) depends only on v_x: $\partial\dot{x}/\partial v_x = 1$, and every other entry in this row is zero since $\dot{x}$ does not depend on x, y, θ, or v_y.

Row 2 ($\dot{y} = v_y$) is identical in structure: $\partial\dot{y}/\partial v_y = 1$, all other entries zero.

Row 3 ($\dot{\theta} = \omega_{\text{imu}}$) contains no state variable at all on its right-hand side – ω_{imu} is a control input, not a function of x, y, θ, v_x, v_y – so every entry of this row is exactly zero.

Rows 4–5 depend, among the state variables, only on θ (neither depends on x, y, v_x, or v_y). Differentiating the $\dot{v}_x, \dot{v}_y$ equations of §VIII-C with respect to θ and evaluating at $\hat{\theta}$,

$$\frac{\partial \dot{v}_x}{\partial \theta} = -a_x^{(b)} \sin\theta - a_y^{(b)} \cos\theta = -a_y \tag{18}$$

$$\frac{\partial \dot{v}_y}{\partial \theta} = a_x^{(b)} \cos\theta - a_y^{(b)} \sin\theta = a_x \tag{19}$$

with every other entry in rows 4–5 equal to zero. Substituting these values into the template above completes the proof. This reproduces exactly the coefficient of $\delta\theta$ obtained by hand in §VIII-D.2, confirming that the informal worked example and the formal partial-derivative calculation agree, as they must.

The entire nonlinearity of the system is concentrated in the two entries $-a_y, a_x$ of column 3 of A: a small heading error rotates the applied acceleration vector and injects a velocity error, exactly as the worked example showed for the $\dot{v}_x$ row. Every other entry of A is constant, independent of the operating point – rows 1, 2, and 3 contain no trigonometric functions at all, since $\dot{x}, \dot{y}, \dot{\theta}$ never involved a rotation.

5) The Input Jacobian B

The same template-then-differentiate procedure is applied with columns ordered over the input $(a_x^{(b)}, a_y^{(b)}, \omega_{\text{imu}})$ rather than the state.

[Continuous Jacobian B] Writing out every partial-derivative entry individually before evaluating any of them, the template is

$$B = \begin{bmatrix} \frac{\partial \dot{x}}{\partial a_x^{(b)}} & \frac{\partial \dot{x}}{\partial a_y^{(b)}} & \frac{\partial \dot{x}}{\partial \omega_{\text{imu}}} \\ \frac{\partial \dot{y}}{\partial a_x^{(b)}} & \frac{\partial \dot{y}}{\partial a_y^{(b)}} & \frac{\partial \dot{y}}{\partial \omega_{\text{imu}}} \\ \frac{\partial \dot{\theta}}{\partial a_x^{(b)}} & \frac{\partial \dot{\theta}}{\partial a_y^{(b)}} & \frac{\partial \dot{\theta}}{\partial \omega_{\text{imu}}} \\ \frac{\partial \dot{v}_x}{\partial a_x^{(b)}} & \frac{\partial \dot{v}_x}{\partial a_y^{(b)}} & \frac{\partial \dot{v}_x}{\partial \omega_{\text{imu}}} \\ \frac{\partial \dot{v}_y}{\partial a_x^{(b)}} & \frac{\partial \dot{v}_y}{\partial a_y^{(b)}} & \frac{\partial \dot{v}_y}{\partial \omega_{\text{imu}}} \end{bmatrix}. \tag{20}$$

Evaluating each of these fifteen entries (proof below) and substituting gives the Control (Input) Jacobian,

$$\boxed{B = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \\ \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \end{bmatrix}}. \tag{21}$$

Proof: the position equations of §VIII-C do not involve the input at all, so rows 1–2 vanish entirely. Row 3 gives $\partial\dot{\theta}/\partial\omega_{\text{imu}} = 1$ directly, with the remaining two entries of that row zero, since $\dot{\theta}$ does not depend on $a_x^{(b)}$ or $a_y^{(b)}$. Differentiating the $\dot{v}_x, \dot{v}_y$ equations with respect to $a_x^{(b)}, a_y^{(b)}$ gives rows 4–5 directly – these derivatives simply reproduce the entries of the rotation matrix $R(\theta)$, since $\dot{\mathbf{v}} = R(\theta)\mathbf{a}^{(b)}$ is already linear in $\mathbf{a}^{(b)}$ before any approximation – with no dependence on ω_{imu}.

Unlike A, whose only nonzero trigonometric entries came from differentiating a nonlinear function, the entries of B are trigonometric simply because the underlying map $\mathbf{a}^{(b)} \mapsto \dot{\mathbf{v}}$ was already linear (a rotation is a linear map). No Taylor expansion was needed to obtain B; differentiating a linear function of the input simply reproduces its (constant, given $\hat{\theta}$) coefficient matrix.

With A and B so constructed, the continuous-time linear error-state system of §VIII-D.1 is fully specified. It is worth emphasizing that this system is not itself used to propagate the state estimate – the exact nonlinear kinematics of §VIII-C are used for that purpose, since they involve no approximation and are no more costly to evaluate. Instead, it exists solely to be discretized into a form that permits closed-form propagation of the estimation error covariance, developed next.

E. Discretization and Prediction Step

1) Euler Discretization Principle

The simplest rule for approximating how a quantity changes over a short time step Δt, given only its instantaneous rate of change, truncates the Taylor expansion of $x(t + \Delta t)$ at first order:

$$\mathbf{x}[k] \approx \mathbf{x}[k-1] + \dot{\mathbf{x}}[k-1]\,\Delta t. \tag{22}$$

This approximates the true (curved) trajectory over the interval by a straight line, using the slope $\dot{\mathbf{x}}[k-1]$ known at the start of the interval – the same first-order truncation idea used already in §VIII-D, only now the expansion variable is time, Δt, rather than a state perturbation, $\delta\theta$.

Applying this rule to the linear error-state system gives the discrete-time state-transition and control matrices

$$\Phi \approx I_5 + A\Delta t, \qquad \Gamma \approx B\Delta t, \tag{23}$$

where I_5 is the 5×5 identity matrix. Substituting the continuous Jacobians of §VIII-D.4–VIII-D.5 gives the Discrete-Time State-Transition Matrix,

$$\boxed{\Phi = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & 0 & \Delta t \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -a_y\Delta t & 1 & 0 \\ 0 & 0 & a_x\Delta t & 0 & 1 \end{bmatrix}}, \tag{24}$$

and the Discrete-Time Control Matrix,

$$\boxed{\Gamma = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \Delta t \\ \Delta t\cos\theta & -\Delta t\sin\theta & 0 \\ \Delta t\sin\theta & \Delta t\cos\theta & 0 \end{bmatrix}}. \tag{25}$$

Both matrices must be recomputed at every prediction cycle from the latest heading estimate and IMU sample, since θ, a_x, a_y all evolve as the vehicle moves – the LTV behavior already flagged in §VIII-D.

2) Nonlinear State Prediction

It bears repeating: the predicted state itself is not computed using Φ. Applying the Euler rule directly to the exact nonlinear kinematics of §VIII-C, rather than to the linearized error system, the predicted state $\hat{\mathbf{x}}_{k|k-1}$ is obtained in two vector-valued pieces. The position block integrates velocity directly, with no rotation involved:

$$\begin{bmatrix} \hat{x}_{k|k-1} \\ \hat{y}_{k|k-1} \end{bmatrix} = \begin{bmatrix} \hat{x}_{k-1|k-1} \\ \hat{y}_{k-1|k-1} \end{bmatrix} + \begin{bmatrix} \hat{v}_{x,k-1|k-1} \\ \hat{v}_{y,k-1|k-1} \end{bmatrix} \Delta t. \tag{26}$$

The velocity block integrates the rotated accelerometer reading, using $R(\hat{\theta}_{k-1|k-1})$, evaluated at the current heading estimate:

$$\begin{bmatrix} \hat{v}_{x,k|k-1} \\ \hat{v}_{y,k|k-1} \end{bmatrix} = \begin{bmatrix} \hat{v}_{x,k-1|k-1} \\ \hat{v}_{y,k-1|k-1} \end{bmatrix} + R(\hat{\theta}_{k-1|k-1}) \begin{bmatrix} a_x^{(b)} \\ a_y^{(b)} \end{bmatrix} \Delta t. \tag{27}$$

Written out in scalar components:

$$\hat{x}_{k|k-1} = \hat{x}_{k-1|k-1} + \hat{v}_{x,k-1|k-1}\Delta t, \tag{28}$$
$$\hat{y}_{k|k-1} = \hat{y}_{k-1|k-1} + \hat{v}_{y,k-1|k-1}\Delta t, \tag{29}$$
$$\hat{v}_{x,k|k-1} = \hat{v}_{x,k-1|k-1} + a_x\Delta t, \tag{30}$$
$$\hat{v}_{y,k|k-1} = \hat{v}_{y,k-1|k-1} + a_y\Delta t, \tag{31}$$

with a_x, a_y evaluated at $\hat{\theta}_{k-1|k-1}$ per §VIII-C. The heading block is a scalar and requires wrapping back into $(-\pi, \pi]$:

$$\hat{\theta}_{k|k-1} = \operatorname{atan2}(\sin\Theta, \cos\Theta), \quad \Theta \triangleq \hat{\theta}_{k-1|k-1} + \omega_{\text{imu}}\Delta t. \tag{32}$$

The predicted state uses the exact nonlinear model, not the linearized Φ; the latter is reserved for covariance propagation alone (§VIII-E.3). There is no contradiction in using both the exact nonlinear equations and their linearization within the same algorithm: the nonlinear equations are exact and cheap to evaluate, so they are used wherever a definite numerical answer is needed (the state); the linearization is needed only where an exact closed-form propagation rule is required for a statistical quantity (the covariance), which has no nonlinear analogue. A plain sum Θ may fall outside $(-\pi, \pi]$ after repeated integration around full turns; the two-argument arctangent recovers the unique equivalent angle from Θ's sine and cosine, since both are periodic in Θ with period 2π.

3) Covariance Prediction

Unlike the state estimate, the covariance matrix P – which quantifies how uncertain the filter is about its own estimate – cannot be propagated in closed form through a nonlinear map. This is precisely why the linearization of §VIII-D was necessary: it supplies Φ for the standard linear covariance recursion,

$$\boxed{P_{k|k-1} = \Phi P_{k-1|k-1}\Phi^\top + Q_k}. \tag{33}$$

In words: the new uncertainty equals the old uncertainty, rotated and scaled by however the linearized dynamics stretch errors over one step ($\Phi P \Phi^\top$), plus whatever additional uncertainty the process noise injects over that step (Q_k).

Process Noise Covariance Q_k has two additive contributions. IMU-input noise is mapped through the control Jacobian Γ – sensor noise on the accelerometer and gyroscope is propagated through exactly the same linear map that propagates the nominal input into a state change, since to first order, noise on the input perturbs the state in the same way the input itself does. The Input Noise Covariance is

$$\boxed{Q_u = \begin{bmatrix} \sigma_{a_x}^2 & 0 & 0 \\ 0 & \sigma_{a_y}^2 & 0 \\ 0 & 0 & \sigma_\omega^2 \end{bmatrix}, \qquad Q_{\text{IMU}} = \Gamma Q_u \Gamma^\top}. \tag{34}$$

For the IMU characterized in this study, the accelerometer noise variance is taken as $\sigma_{a_x}^2 = \sigma_{a_y}^2 = 0.09\ (\text{m/s}^2)^2$, and the gyroscope noise variance as $\sigma_\omega^2 = 9.61\,(^\circ/\text{s})^2 \Rightarrow \sigma_\omega^2 \approx 2.928 \times 10^{-3}\,\text{rad}^2/\text{s}^2$, the conversion following from $1^\circ = \pi/180$ rad, so that $\sigma_\omega^2[\text{rad}^2/\text{s}^2] = \sigma_\omega^2[(^\circ/\text{s})^2] \times (\pi/180)^2$. Because the state vector carries θ in radians, the converted value is the one substituted into Q_u above; the datasheet value in $(^\circ/\text{s})^2$ is retained alongside it for traceability to the sensor specification.

A second, additive term captures unmodeled disturbance not explained by IMU noise (e.g. wheel slip, unmodeled dynamics), the Baseline Process Noise Covariance,

$$\boxed{Q_{\text{base}} = \Delta t \cdot \text{diag}(q_{\text{scale}}q_x,\ q_{\text{scale}}q_y,\ q_\theta,\ q_{v_x},\ q_{v_y})}, \tag{35}$$

where $q_{\text{scale}} \geq 1$ is raised during intervals of dead reckoning without UWB correction, so the filter's admitted uncertainty grows appropriately faster and remains statistically consistent in the absence of a correction – the filter honestly reports that it is "less sure" of its position while it has gone without an external fix. The total Process Noise Covariance is

$$\boxed{Q_k = Q_{\text{IMU}} + Q_{\text{base}}}. \tag{36}$$

F. Measurement Model

§VIII-E completed the prediction half of the filter, which runs at every IMU tick but, being open-loop dead reckoning, drifts without bound if never corrected. This subsection builds the measurement model needed for the correction step of §VIII-G.

1) Resolving the Multilateration Nonlinearity Upstream

A single UWB anchor measures only a scalar range to the vehicle; no individual anchor measures (x, y) directly. Combining ranges from three or more anchors into a Cartesian position fix – multilateration – is an inherently nonlinear geometric operation, analogous to locating a point on a map purely from its distances to several known landmarks (this is exactly the role of the geometric solvers of Section V). If this nonlinear step were folded directly into the EKF's measurement equation, the output matrix would itself require a further linearization, of the same flavor as §VIII-D but for the correction step.

Under Assumption A7, this complication is avoided entirely: the multilateration nonlinearity is resolved once, upstream of the EKF, by the dedicated geometric solvers of Section V, so the filter consumes an already-Cartesian position time series. This is what allows the output matrix derived below to be exactly linear, with no approximation required in the correction step.

2) Derived Pseudo-Measurements

Let $(x_{\text{uwb},k}, y_{\text{uwb},k})$ denote the upstream solver's position fix at cycle k, with propagated covariance $P_{\text{uwb},k}$, and $\Delta t_{\text{uwb}} = t_k - t_{k-1}$ the interval between consecutive UWB fixes (generally slower than the IMU tick rate). A position fix alone constrains only two of the five state components; velocity and heading pseudo-measurements are additionally derived from consecutive fixes, so that every state component receives some form of

direct correction:

$$v_{x,\mathrm{uwb},k} = \frac{x_{\mathrm{uwb},k} - x_{\mathrm{uwb},k-1}}{\Delta t_{\mathrm{uwb}}}, \tag{37}$$

$$v_{y,\mathrm{uwb},k} = \frac{y_{\mathrm{uwb},k} - y_{\mathrm{uwb},k-1}}{\Delta t_{\mathrm{uwb}}}, \tag{38}$$

$$\theta_{\mathrm{uwb},k} = \mathrm{atan2}(v_{y,\mathrm{uwb},k}, v_{x,\mathrm{uwb},k}). \tag{39}$$

This last equation is the vehicle's course-over-ground – the direction the vehicle's position is actually moving, inferred from two consecutive fixes – and it approximates yaw only under Assumption A8 (negligible sideslip). It is computed only above the minimum-speed threshold $v_{\min}$ of A9, since dividing two near-zero position differences by one another yields a numerically meaningless direction.

3) Measurement Equation

The pseudo-measurement vector collects all four quantities above alongside the position fix itself:

$$\mathbf{y}_k = [x_{\mathrm{uwb},k} \;\; y_{\mathrm{uwb},k} \;\; \theta_{\mathrm{uwb},k} \;\; v_{x,\mathrm{uwb},k} \;\; v_{y,\mathrm{uwb},k}]^\top, \tag{40}$$

which plays the role of the output $\mathbf{y}$ in the general LTI output equation. The complete measurement equation is therefore

$$\mathbf{y}_k = C\,\mathbf{x}_k + D\,\mathbf{u}_k, \tag{41}$$

where the uncertainty in the pseudo-measurement itself is accounted for entirely through the measurement noise covariance R_k, constructed explicitly in §VIII-F.5 and used directly in the correction step of §VIII-G. The remainder of this subsection derives C and D.

[Output Matrix C] Let $C \triangleq \partial\mathbf{y}/\partial\mathbf{x}$, with rows ordered over the components of $\mathbf{y}_k$ above and columns ordered over the state (x, y, θ, v_x, v_y). Writing out every partial-derivative entry individually, before evaluating any of them, the template is

$$C = \begin{bmatrix} \frac{\partial x_{\mathrm{uwb}}}{\partial x} & \frac{\partial x_{\mathrm{uwb}}}{\partial y} & \frac{\partial x_{\mathrm{uwb}}}{\partial \theta} & \frac{\partial x_{\mathrm{uwb}}}{\partial v_x} & \frac{\partial x_{\mathrm{uwb}}}{\partial v_y} \\ \frac{\partial y_{\mathrm{uwb}}}{\partial x} & \frac{\partial y_{\mathrm{uwb}}}{\partial y} & \frac{\partial y_{\mathrm{uwb}}}{\partial \theta} & \frac{\partial y_{\mathrm{uwb}}}{\partial v_x} & \frac{\partial y_{\mathrm{uwb}}}{\partial v_y} \\ \frac{\partial \theta_{\mathrm{uwb}}}{\partial x} & \frac{\partial \theta_{\mathrm{uwb}}}{\partial y} & \frac{\partial \theta_{\mathrm{uwb}}}{\partial \theta} & \frac{\partial \theta_{\mathrm{uwb}}}{\partial v_x} & \frac{\partial \theta_{\mathrm{uwb}}}{\partial v_y} \\ \frac{\partial v_{x,\mathrm{uwb}}}{\partial x} & \frac{\partial v_{x,\mathrm{uwb}}}{\partial y} & \frac{\partial v_{x,\mathrm{uwb}}}{\partial \theta} & \frac{\partial v_{x,\mathrm{uwb}}}{\partial v_x} & \frac{\partial v_{x,\mathrm{uwb}}}{\partial v_y} \\ \frac{\partial v_{y,\mathrm{uwb}}}{\partial x} & \frac{\partial v_{y,\mathrm{uwb}}}{\partial y} & \frac{\partial v_{y,\mathrm{uwb}}}{\partial \theta} & \frac{\partial v_{y,\mathrm{uwb}}}{\partial v_x} & \frac{\partial v_{y,\mathrm{uwb}}}{\partial v_y} \end{bmatrix}. \tag{42}$$

Evaluating each of these twenty-five entries (proof below) and substituting gives the Output (Measurement) Matrix,

$$\boxed{C = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} = I_5}. \tag{43}$$

Proof: by construction in §VIII-F.2 and Assumption A7, each component of $\mathbf{y}_k$ is a direct, one-to-one estimate of the corresponding state component, with no intervening nonlinear transformation left inside the filter – the multilateration nonlinearity having already been resolved upstream. That is, $x_{\mathrm{uwb}} = x$, $y_{\mathrm{uwb}} = y$, $\theta_{\mathrm{uwb}} = \theta$, $v_{x,\mathrm{uwb}} = v_x$, $v_{y,\mathrm{uwb}} = v_y$ hold identically. Differentiating each row of the template above with respect to each state variable in turn:

Row 1 ($x_{\mathrm{uwb}} = x$): $\partial x_{\mathrm{uwb}}/\partial x = 1$; the derivative with respect to y, θ, v_x, v_y is 0 in each case, since x_{uwb} carries no dependence on them.

Row 2 ($y_{\mathrm{uwb}} = y$): $\partial y_{\mathrm{uwb}}/\partial y = 1$; all other entries in this row are 0.

Row 3 ($\theta_{\mathrm{uwb}} = \theta$): $\partial \theta_{\mathrm{uwb}}/\partial \theta = 1$; all other entries in this row are 0.

Row 4 ($v_{x,\mathrm{uwb}} = v_x$): $\partial v_{x,\mathrm{uwb}}/\partial v_x = 1$; all other entries in this row are 0.

Row 5 ($v_{y,\mathrm{uwb}} = v_y$): $\partial v_{y,\mathrm{uwb}}/\partial v_y = 1$; all other entries in this row are 0.

Substituting these twenty-five evaluated entries gives $C = I_5$ exactly – unlike the state Jacobian A of §VIII-D.4, no entry here involves θ nonlinearly, because the output equation performs no rotation. This is also unlike A and B in one further respect: C is not a Jacobian evaluated at an operating point that would change from cycle to cycle – it is exact and constant at every operating point, because the underlying map from state to output is already linear.

[Direct Feedthrough Matrix D] Let $D \triangleq \partial\mathbf{y}/\partial\mathbf{u}$, with rows again ordered over the components of $\mathbf{y}_k$ and columns ordered over the input $(a_x^{(b)}, a_y^{(b)}, \omega_{\mathrm{imu}})$. Writing out every partial-derivative entry individually, before evaluating any of them, the template is

$$D = \begin{bmatrix} \frac{\partial x_{\mathrm{uwb}}}{\partial a_x^{(b)}} & \frac{\partial x_{\mathrm{uwb}}}{\partial a_y^{(b)}} & \frac{\partial x_{\mathrm{uwb}}}{\partial \omega_{\mathrm{imu}}} \\ \frac{\partial y_{\mathrm{uwb}}}{\partial a_x^{(b)}} & \frac{\partial y_{\mathrm{uwb}}}{\partial a_y^{(b)}} & \frac{\partial y_{\mathrm{uwb}}}{\partial \omega_{\mathrm{imu}}} \\ \frac{\partial \theta_{\mathrm{uwb}}}{\partial a_x^{(b)}} & \frac{\partial \theta_{\mathrm{uwb}}}{\partial a_y^{(b)}} & \frac{\partial \theta_{\mathrm{uwb}}}{\partial \omega_{\mathrm{imu}}} \\ \frac{\partial v_{x,\mathrm{uwb}}}{\partial a_x^{(b)}} & \frac{\partial v_{x,\mathrm{uwb}}}{\partial a_y^{(b)}} & \frac{\partial v_{x,\mathrm{uwb}}}{\partial \omega_{\mathrm{imu}}} \\ \frac{\partial v_{y,\mathrm{uwb}}}{\partial a_x^{(b)}} & \frac{\partial v_{y,\mathrm{uwb}}}{\partial a_y^{(b)}} & \frac{\partial v_{y,\mathrm{uwb}}}{\partial \omega_{\mathrm{imu}}} \end{bmatrix}. \tag{44}$$

Evaluating each of these fifteen entries (proof below) and substituting gives the Direct Feedthrough Matrix,

$$\boxed{D = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = 0_{5\times 3}}. \tag{45}$$

Proof: every pseudo-measurement in §VIII-F.2 is built purely from consecutive upstream UWB position fixes

$(x_{\mathrm{uwb},k-1}, y_{\mathrm{uwb},k-1}), (x_{\mathrm{uwb},k}, y_{\mathrm{uwb},k})$ and the sampling interval Δt_{uwb}; none of the five rows of $\mathbf{y}_k$ contains the current IMU input $\mathbf{u}_k = [a_x^{(b)}, a_y^{(b)}, \omega_{\mathrm{imu}}]^\top$ at all. Differentiating each row of the template above with respect to each of the three input components therefore gives 0 identically, in every one of the fifteen entries, with no cancellation or approximation involved – the entries are zero because the input simply never appears in the defining equations, not because of any linearization. Substituting these fifteen evaluated entries gives $D = 0_{5\times 3}$ exactly.

This is the point in the derivation at which every state component acquires a direct measurement channel. Heading, in particular, receives its only closed-loop correction here, since the process model of §VIII-C contains no self-correcting feedback for θ – row 3 of A in §VIII-D.4 is entirely zero, meaning nothing in the prediction step ever reduces heading uncertainty on its own. Any row of $\mathbf{y}_k$ may be dropped on a given cycle if unavailable (e.g., the very first fix, or a cycle below the speed threshold), in which case C, D, and $\mathbf{y}_k$ simply reduce to the corresponding sub-block, with no other change to the algorithm.

4) Measurement Noise

Under Assumption A10, the five pseudo-measurement rows are treated as mutually independent, so that the Measurement Noise Covariance R_k, written out in full as the complete 5×5 matrix, is

$$\boxed{R_k = \begin{bmatrix} [P_{\mathrm{uwb},k}]_{11} & [P_{\mathrm{uwb},k}]_{12} & 0 & 0 & 0 \\ [P_{\mathrm{uwb},k}]_{21} & [P_{\mathrm{uwb},k}]_{22} & 0 & 0 & 0 \\ 0 & 0 & \sigma_\theta^2 & 0 & 0 \\ 0 & 0 & 0 & \sigma_{v_x}^2 & 0 \\ 0 & 0 & 0 & 0 & \sigma_{v_y}^2 \end{bmatrix}}. \tag{46}$$

The top-left 2×2 block is the propagated covariance of the upstream UWB position fix itself; the remaining three diagonal entries are the heading and velocity pseudo-measurement variances, with all cross-block entries zero under A10. Grouping the position block back into $P_{\mathrm{uwb},k}$ and the velocity block into R_v, this is written more compactly as

$$\boxed{R_k = \begin{bmatrix} P_{\mathrm{uwb},k} & 0 & 0 \\ 0^\top & \sigma_\theta^2 & 0^\top \\ 0 & 0 & R_v \end{bmatrix}, \quad R_v = \begin{bmatrix} \sigma_{v_x}^2 & 0 \\ 0 & \sigma_{v_y}^2 \end{bmatrix}}, \tag{47}$$

$$\sigma_{v_x}^2, \sigma_{v_y}^2 \approx \frac{2\,[P_{\mathrm{uwb},k}]_{ii}}{\Delta t_{\mathrm{uwb}}^2}. \tag{48}$$

This last relation follows from standard error propagation through the finite difference used in §VIII-F.2: since velocity is a difference of two independent noisy position fixes, each contributing variance $[P_{\mathrm{uwb},k}]_{ii}$, the variance of the difference is the sum of the two (hence the factor of 2), scaled by $1/\Delta t_{\mathrm{uwb}}^2$ from the division. Because of this $1/\Delta t_{\mathrm{uwb}}^2$ scaling, the velocity pseudo-measurement's variance grows quadratically as Δt_{uwb} shrinks – fixes taken close together in time produce noisier velocity estimates, not cleaner ones – so this pseudo-measurement should be weighted, and gated, conservatively.

G. Correction (Update) Step

With C linear and constant (§VIII-F), the correction reduces to the standard EKF update equations, explained below in execution order.

1) Predicted Measurement and Innovation

Given the predicted state $\hat{\mathbf{x}}_{k|k-1}$ from §VIII-E, the filter first asks: if the current prediction were exactly correct, what would the UWB system have reported? This predicted measurement, and the discrepancy (innovation, or residual) between it and what was actually observed, are

$$\hat{\mathbf{y}}_{k|k-1} = C\,\hat{\mathbf{x}}_{k|k-1}, \qquad \mathbf{e}_k = \mathbf{y}_k - \hat{\mathbf{y}}_{k|k-1}, \tag{49}$$

with the heading component of $\mathbf{e}_k$ wrapped through the same `atan2` construction as in §VIII-E.2, since a difference of two angles can likewise fall outside $(-\pi, \pi]$ (e.g., a true heading change from 179° to -179° is a 2° turn, not a 358° one).

2) Innovation Covariance

The innovation $\mathbf{e}_k$ is itself uncertain, for two reasons: the prior state estimate was uncertain, and the measurement itself is noisy. Both sources are combined into the Innovation Covariance:

$$\boxed{S_k = C\,P_{k|k-1}\,C^\top + R_k}. \tag{50}$$

3) Kalman Gain

The Kalman gain is the matrix that minimizes the trace (total variance) of the posterior covariance $P_{k|k}$ over all linear correction rules of the form $\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k\mathbf{e}_k$; its closed-form solution is

$$\boxed{K_k = P_{k|k-1}\,C^\top\,S_k^{-1}}. \tag{51}$$

Qualitatively, K_k is large where the prior uncertainty $P_{k|k-1}$ is large relative to the measurement noise R_k (favoring the new measurement), and small in the opposite case (favoring the prior estimate); it performs this trade-off jointly and optimally across all five state components at once, rather than treating each independently.

4) State and Covariance Update

The state and covariance are updated as

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k\,\mathbf{e}_k, \tag{52}$$

with the heading component re-wrapped via `atan2` exactly as in §VIII-E.2, and the Updated (A Posteriori) Error Covariance, in the numerically robust Joseph form:

$$\boxed{P_{k|k} = (I_5 - K_kC)\,P_{k|k-1}\,(I_5 - K_kC)^\top + K_k\,R_k\,K_k^\top}. \tag{53}$$

This form is used in place of the algebraically equivalent, shorter expression $P_{k|k} = (I_5 - K_k C)P_{k|k-1}$ because it guarantees a symmetric, positive semi-definite result even under finite-precision rounding error – a practical concern for any long-running embedded implementation, where small arithmetic errors accumulate over thousands of cycles.

An innovation gate (a Mahalanobis test on $\mathbf{e}_k^\top S_k^{-1}\mathbf{e}_k$, applied per-row or jointly) is typically evaluated before the state and covariance update are accepted, to reject outlying UWB fixes – e.g., a single anomalous range reading caused by a reflection off a wall – before they are allowed to corrupt the state estimate.

H. Algorithm Summary

Algorithm 1 summarizes one complete prediction–correction cycle, collecting every equation derived in §VIII-E–VIII-G into execution order. Steps 1–3 execute at every IMU tick; Steps 4–8 execute only when a fresh UWB fix is available, typically less frequent than the IMU rate.

Table 10: Algorithm 1 – Single-Cycle EKF Prediction–Correction Loop

1: Acquire IMU sample $\mathbf{u}_k = [a_x^{(b)}, a_y^{(b)}, \omega_{\text{imu}}]^\top$.
2: Predict state $\hat{\mathbf{x}}_{k|k-1}$ using §VIII-E.2 (exact nonlinear model).
3: Predict covariance $P_{k|k-1}$ using §VIII-E.1, VIII-E.3.
4: **if** a new UWB fix is available **then**
5: Form pseudo-measurement $\mathbf{y}_k$ using §VIII-F.2–VIII-F.3.
6: Compute innovation $\mathbf{e}_k$ and covariance S_k using §VIII-G.1–VIII-G.2.
7: Compute Kalman gain K_k using §VIII-G.3.
8: Update $\hat{\mathbf{x}}_{k|k}$, $P_{k|k}$ using §VIII-G.4.
9: **else**
10: Set $\hat{\mathbf{x}}_{k|k} \leftarrow \hat{\mathbf{x}}_{k|k-1}$, $P_{k|k} \leftarrow P_{k|k-1}$.
11: **end if**

I. Discussion

Three approximations warrant emphasis beyond what has already been noted inline, since each represents a genuine limitation rather than a simplification with no cost.

Correlated pseudo-measurement noise. The block-diagonal R_k of §VIII-F.5 neglects genuine correlation: x_{uwb}, θ_{uwb}, and $v_{x,\text{uwb}}, v_{y,\text{uwb}}$ all derive from the same short window of noisy position fixes, so treating their errors as independent, as A10 does, is optimistic; a more faithful model would carry off-diagonal covariance terms linking these rows.

Sideslip. Assumption A8 (course-over-ground as yaw) fails whenever sideslip is non-negligible – for example during sharp turns or on low-friction surfaces, where the vehicle's direction of travel and its direction of pointing diverge. The heading pseudo-measurement of §VIII-F.2 should therefore be understood as approximate rather than exact under such conditions, and a filter operating in such regimes may need an explicit sideslip estimate to remain accurate.

Unchecked heading drift at low speed. Because the process model of §VIII-C provides no self-correcting feedback for θ (row 3 of A in §VIII-D.4 is identically zero), the UWB-derived heading of §VIII-F.2 is the sole mechanism by which a drifting gyroscope bias can be corrected. This correction is unavailable during any interval in which the vehicle speed falls below $v_{\min}$ (A9), so heading drift accumulates unchecked in exactly such intervals – a vehicle that stops moving, even briefly, loses its only source of heading correction until it moves again.

J. Section Conclusion

Starting from the continuous-time nonlinear kinematics of a wheeled vehicle, this section derived a five-state EKF whose sole nonlinearity is the heading-dependent rotation of body-frame acceleration into the world frame, following a systematic and reusable template-then-differentiate procedure for constructing each Jacobian – including, in §VIII-D, an explicit term-by-term Taylor expansion showing how the linear error-state equation $\dot{\delta\mathbf{x}} = A\delta\mathbf{x} + B\delta\mathbf{u}$ arises from first principles, and a direct explanation of why the trigonometric functions $\sin\theta, \cos\theta$ persist after linearization: they are evaluated at the known current estimate $\hat{\theta}$, not approximated, and it is only the unknown error $\delta\theta$ that is ever treated as small. On the correction side, resolving UWB multilateration upstream of the filter, rather than inside it, yields an exactly linear, fully observable measurement model, at the cost of a compounding noise penalty on the derived heading and velocity rows that must be reflected honestly in R_k. Every step of the derivation – linearization, discretization, and the statistical construction of the Kalman gain – was developed from the underlying calculus and linear algebra, so that the complete filter can be reconstructed by a reader without prior exposure to Kalman filtering [2, 1, 3].

K. Notation Reference

Table 11: Notation used throughout Section VIII

Symbol	Meaning
Coordinate frames	
World frame	Fixed frame coincident with the UWB anchor constellation
Body frame, (b)	Frame attached to, and rotating with, the vehicle chassis
State, error, and estimate notation	
$\mathbf{x}_k \in \mathbb{R}^5$	True state at discrete time step k
$\hat{\mathbf{x}}_{k\|k-1}$	Predicted (a priori) state estimate
$\hat{\mathbf{x}}_{k\|k}$	Corrected (a posteriori) state estimate
$\delta\mathbf{x} \triangleq \mathbf{x} - \hat{\mathbf{x}}$	State estimation error
$\hat{(\cdot)}$	Filter estimate of the quantity underneath
$\delta(\cdot)$	Unknown error between a true quantity and its estimate
State vector components	
x, y	Vehicle position, world frame (m)
θ	Vehicle heading (yaw), rad
v_x, v_y	Vehicle linear velocity, world frame (m/s)
Control input components	
$\mathbf{u}_k \in \mathbb{R}^3$	Control input vector
$a_x^{(b)}, a_y^{(b)}$	Body-frame accelerometer readings (m/s^2)
ω_{imu}	Gyroscope-measured yaw rate (rad/s)
a_x, a_y	World-frame acceleration $= R(\theta)[a_x^{(b)}, a_y^{(b)}]^\top$
Noise parameters	
$\sigma^2_{a_x}, \sigma^2_{a_y}$	Accelerometer noise variance (m/s^2)^2
σ^2_ω	Gyroscope noise variance rad^2/s^2
σ^2_θ	Heading pseudo-measurement noise variance (rad^2)
$\sigma^2_{v_x}, \sigma^2_{v_y}$	Velocity pseudo-measurement noise variance (m/s)^2
q_{scale}	Dead-reckoning noise inflation factor, ≥ 1
$q_x, q_y, q_\theta, q_{v_x}, q_{v_y}$	Baseline per-state process noise density
Matrices	
$A \in \mathbb{R}^{5\times5}$	Continuous-time state (process) Jacobian
$B \in \mathbb{R}^{5\times3}$	Continuous-time input (control) Jacobian
$\Phi \in \mathbb{R}^{5\times5}$	Discrete-time state-transition matrix
$\Gamma \in \mathbb{R}^{5\times3}$	Discrete-time control matrix
$C \in \mathbb{R}^{5\times5}$	Output (measurement) matrix; $C = I_5$
$D \in \mathbb{R}^{5\times3}$	Direct feedthrough matrix; $D = 0$ throughout
Q_k, R_k	Process / measurement noise covariance
$Q_{\text{IMU}}, Q_{\text{base}}$	IMU-induced / baseline contributions to Q_k
$R(\theta) \in \mathbb{R}^{2\times2}$	Planar rotation matrix, body-to-world
$P_{k\|k-1}, P_{k\|k}$	A priori / a posteriori state covariance
$P_{\text{uwb},k}$	Covariance of the upstream UWB position fix
S_k	Innovation covariance
K_k	Kalman gain
I_n	$n \times n$ identity matrix
Measurement quantities	
$\mathbf{y}_k \in \mathbb{R}^5$	Pseudo-measurement (output) vector at step k
$x_{\text{uwb},k}, y_{\text{uwb},k}$	Upstream-multilateration Cartesian position fix
$\theta_{\text{uwb},k}$	Course-over-ground pseudo-heading
$v_{x,\text{uwb},k}, v_{y,\text{uwb},k}$	Finite-difference velocity pseudo-measurement
$\mathbf{e}_k$	Innovation (measurement residual)
Scalars, parameters, operators	
$\Delta t \triangleq t_k - t_{k-1}$	IMU sampling interval (s)
Δt_{uwb}	Interval between consecutive UWB fixes (s)
$v_{\min}$	Minimum-speed gating threshold (m/s)
$\text{atan2}(\cdot,\cdot)$	Four-quadrant inverse tangent, range $(-\pi, \pi]$
$\mathcal{N}(\mu, \Sigma)$	Gaussian distribution, mean μ, covariance Σ

L. Modeling Assumptions

Table 12: Modeling assumptions used in Section VIII

ID	Statement
A1	Sampling: the prediction step runs at $\Delta t = t_k - t_{k-1}$, the (possibly variable) IMU tick period.
A2	World frame: x, y are expressed in a fixed global frame coincident with the UWB anchor frame; θ is yaw relative to this frame.
A3	Velocity frame: v_x, v_y are world-frame, so Cartesian UWB-derived quantities can correct them directly.
A4	Constant-input-over-step dynamics: linear acceleration and yaw rate are treated as piecewise-constant controls over one prediction step.
A5	First-order (Euler) discretization only; no RK2/RK4 integration is used.
A6	Sensor roles: the IMU enters only the prediction step; the UWB system enters only the correction step.
A7	Upstream multilateration: combining UWB ranges into a Cartesian fix is resolved upstream of the filter (Section V), rendering the output matrix C exactly linear.
A8	Course-over-ground approximates yaw; valid only when sideslip is negligible.
A9	Minimum-speed gating: direction-derived quantities are dropped below $v_{\min}$.
A10	Block-diagonal measurement noise: position, heading, and velocity pseudo-measurements are treated as mutually independent in R_k.
A11	Process and measurement noise are zero-mean, white, Gaussian, and mutually independent; no sensor bias states are carried.
A12	All Jacobians are evaluated at the current state estimate, never at the unknown true state.

IX. Experimental Results

A. Test Setup and Ground-Truth Reference

To evaluate the fused localization output against a known-accurate reference, ground truth was captured using a robotic total station (RTS) rather than a second UWB unit, a tape measure, or visual markers. A robotic total station is a survey-grade instrument that combines an electronic distance meter (EDM) with a motorized, servo-driven telescope: it tracks a retroreflective prism mounted on the moving target and continuously measures the angle and slant range to that prism, converting the pair into a 3D coordinate in its own fixed station frame at each instant. Because the EDM measures range via a modulated infrared/laser carrier rather than radio Time-of-Flight, and the angular encoders resolve sub-arcsecond rotation, a robotic total station in tracking mode is routinely accurate to within a few millimetres in open line-of-sight conditions – roughly an order of magnitude tighter than the UWB system under test. This makes it a suitable "truth" reference for quantifying the residual error of the UWB/IMU fusion stack, rather than just comparing two similarly-noisy sensors against each other.

Two test conditions were run: a *stationary* test, in which the tag was held fixed and the fused output was logged over time to characterize pure noise and static bias; and a *moving* test, in which the tag was carried through a closed-loop path on the warehouse floor while both the RTS and the UWB/EKF stack logged position continuously.

B. Timing Alignment Between Ground Truth and the UWB Estimate

A raw sample-by-sample comparison between the RTS stream and the UWB/EKF stream is not directly meaningful, because the UWB estimate is not instantaneous: by the time a position appears on the operator display, it has already passed through the full pipeline of Section IV – serial acquisition, ML distance correction (Section VI), geometric solving (Section V), EKF fusion (Section VIII), and the Wi-Fi bridge to the remote display – each stage adding a small, roughly fixed processing delay. If this delay is not accounted for, a perfectly accurate but merely *late* UWB estimate would appear as a large position error, when in fact it is a timing offset, not a localization error.

To separate these two effects, the Moving Test results reported below are *timing-corrected*: the constant end-to-end pipeline latency was estimated (via cross-correlation of the UWB and RTS trajectories) and subtracted from the UWB timestamps before computing any error metric, so that each UWB fix is compared against the RTS ground-truth position at the same real-world instant it actually corresponds to, rather than the instant it was logged. This isolates the sensor-fusion and ranging error from pipeline latency, which is a separate, and separately addressable, engineering concern.

C. Trajectory Comparison

Fig. 14 overlays the fused UWB trajectory against the RTS ground truth for the full moving-test loop.

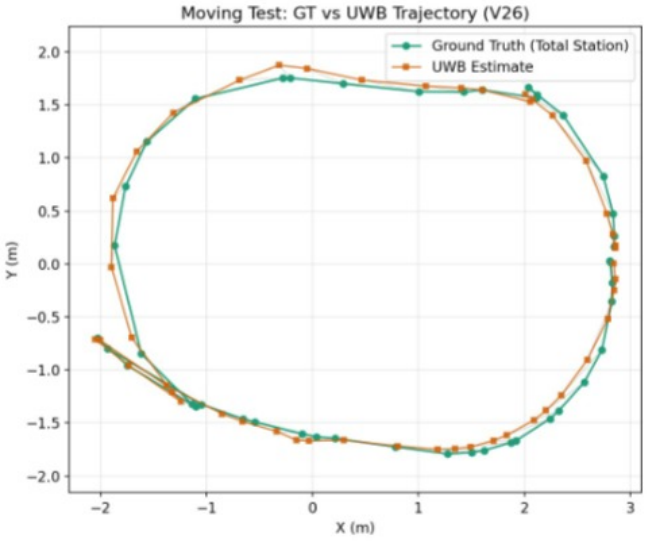


Figure 15: Moving Test: fused UWB trajectory (orange) vs. ground-truth trajectory from the robotic total station (green), V26 pipeline.

The UWB estimate closely tracks the overall shape, size, and closure of the ground-truth loop, with the largest visible deviations occurring around the sharper direction changes on the left-hand side of the path – consistent with the sideslip and heading-drift limitations discussed in §VIII-I, where the EKF's heading correction is weakest during rapid turns.

D. Per-Axis Comparison

Fig. 15 breaks the same run down into its individual X and Y components plotted against time, which makes systematic (bias-like) errors easier to distinguish from transient (noise-like) ones than the trajectory plot alone.

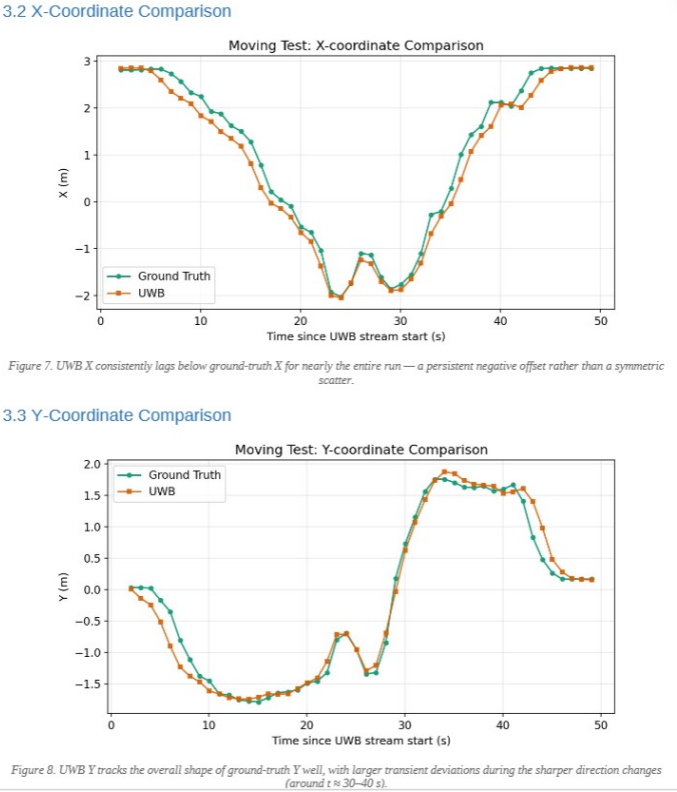


Figure 16: Moving Test: UWB vs. ground-truth X-coordinate (top) and Y-coordinate (bottom) over time.

The X-coordinate plot shows a consistent, persistent negative offset – the UWB estimate lags below ground truth for nearly the entire run, rather than scattering symmetrically around it, pointing to a small systematic bias in that axis rather than pure noise. The Y-coordinate plot tracks the overall shape of the ground-truth curve well, with larger transient deviations appearing during the sharper direction changes around $t \approx 30$–40 s, again consistent with the turning-induced error noted above.

E. Stationary Test Statistics

With the tag held fixed, Table VII and Table VIII summarize the absolute error against the RTS ground truth and the raw spread of the UWB stream about its own mean, respectively.

Separating these two tables matters: Table VIII shows the fused output is very *precise* when stationary (sub-1.5 cm combined spread), while Table VII shows it is not perfectly *accurate* – there

Table 13: Stationary Test: Absolute Error vs. Ground Truth

Metric	X	Y	2D Position
RMSE	2.95 cm	1.12 cm	3.15 cm
Mean error (bias)	2.81 cm	−0.90 cm	3.02 cm
Std. deviation	0.88 cm	0.66 cm	0.91 cm
Max |error|	6.61 cm	2.36 cm	7.02 cm
Min position error	—	—	1.77 cm

Table 14: Stationary Test: Raw UWB Spread (100 Hz stream)

Spread metric	Value
Std. dev. of UWB X about its own mean	1.1 cm
Std. dev. of UWB Y about its own mean	0.6 cm
Combined 2D precision ($\sqrt{\sigma_x^2 + \sigma_y^2}$)	1.2 cm
Mean UWB fix vs. GT (systematic bias)	(2.8, −0.9) cm

is a roughly 3 cm systematic bias, concentrated almost entirely in X (2.8 cm) with much less in Y (−0.9 cm). Because the spread (Table VIII) is small relative to the bias (Table VII), this points to a fixable calibration offset (e.g. anchor survey error or antenna delay miscalibration, §V-A.1) rather than fundamental sensor noise.

F. Moving Test Statistics (Timing-Corrected)

Table IX reports the same error metrics for the moving test, after the pipeline-latency correction of §IX-B has been applied.

Table 15: Moving Test: Absolute Error vs. Ground Truth (Timing-Corrected)

Metric	X	Y	2D Position
RMSE	7.64 cm	6.01 cm	9.72 cm
Mean error (bias)	−4.29 cm	1.75 cm	8.37 cm
Std. deviation	6.38 cm	5.81 cm	5.00 cm
Max |error|	19.32 cm	18.00 cm	22.62 cm
Min position error	—	—	1.45 cm

As expected, error grows substantially from the stationary case (3.15 cm RMSE) to the moving case (9.72 cm RMSE): while stationary, the EKF has no motion-induced IMU integration error and no course-over-ground heading uncertainty (§VIII-H.2) to contend with, whereas in motion both effects are active simultaneously. The sign flip in the X bias (from $+2.81$ cm stationary to -4.29 cm moving) suggests the moving-test error is dominated by direction-dependent dynamic effects – IMU integration drift and turn-induced heading error – rather than by the same static calibration offset seen in the stationary test, which is consistent with the transient deviations visible in Fig. 15 clustering around the sharpest turns rather than being spread uniformly over the whole run.

Note on the timing correction itself: the values in Table IX already have the fixed pipeline latency (serial read → ML correction → geometric solve → EKF update → Wi-Fi transmit → display, §IV) subtracted out, so the reported 2D RMSE of 9.72 cm reflects genuine spatial estimation error at matched time instants, not an artifact of the UWB stream being a few hundred milliseconds behind the ground truth on the display.

X. Conclusion and Future Work

This paper has consolidated the complete design documentation of the Smart Forklift Localization System into a single reference: the physical hardware and wiring (Section III); the ROS 2 node architecture and inter-process data flow, including the non-ROS ML correction backend (Section IV); the geometric UWB multilateration solvers for the two-, three-, and four-anchor cases together with a GDOP-based confidence signal (Section V); a gradient-boosted ensemble model that corrects raw UWB ranges before they reach the filter (Section VI); the ONNX conversion that made that model practical to run in real time on a Raspberry Pi 5 (Section VII); and the five-state Extended Kalman Filter that fuses IMU prediction with UWB-derived correction into a final pose estimate (Section VIII).

Future work includes: populating Section IX with quantitative accuracy results from field trials; implementing the weighted-NLLS, analytic-Jacobian, robust joint-optimization, and covariance-aware disambiguation improvements identified in §V-I; removing the unused `Raw_Sq` feature from the distance-correction model (§VI-H.3); and extending the EKF's heading-correction mechanism to remain robust during extended low-speed or stationary intervals (§VIII-I).

Acknowledgment

This work was carried out as part of an industrial internship at Krishna Defence and Allied Industries, under the mentorship of Prof. Madhu Vadali, Indian Institute of Technology Gandhinagar.

References

[1] Y. Kanayama, Y. Kimura, F. Miyazaki, and T. Noguchi, "A stable tracking control method for an autonomous mobile robot," in *Proc. IEEE Int. Conf. Robotics and Automation*, 1990, pp. 384–389.

[2] G. Welch and G. Bishop, "An introduction to the Kalman filter," Univ. North Carolina at Chapel Hill, Tech. Rep. TR 95-041, 2006.

[3] R. Banavar, "ME 692: Data and Observer Design," Lecture Slides, Indian Institute of Technology Bombay.

[4] W. H. Foy, "Position-location solutions by Taylor-series estimation," *IEEE Trans. Aerospace and Electronic Systems*, vol. AES-12, no. 2, pp. 187–194, Mar. 1976.

[5] K. Levenberg, "A method for the solution of certain non-linear problems in least squares," *Quarterly of Applied Mathematics*, vol. 2, no. 2, pp. 164–168, 1944.

[6] D. W. Marquardt, “An algorithm for least-squares estimation of nonlinear parameters,” *J. Society for Industrial and Applied Mathematics*, vol. 11, no. 2, pp. 431–441, Jun. 1963.

[7] A. H. Sayed, A. Tarighat, and N. Khajehnouri, “Network-based wireless location,” *IEEE Signal Processing Magazine*, vol. 22, no. 4, pp. 24–40, Jul. 2005.

[8] J. Cota-Ruiz, P. Rivas-Perea, E. Sifuentes, and R. Gonzalez-Landaeta, “A low-complexity geometric bilateration method for localization in wireless sensor networks,” *Sensors*, vol. 12, no. 1, pp. 839–869, Jan. 2012.

[9] C.-C. Chen, C.-Y. Chang, and Y.-N. Li, “Range-free localization scheme in wireless sensor networks based on bilateration,” *Int. J. Distributed Sensor Networks*, vol. 2013, art. 620248, 2013.

[10] S. Maranò, W. M. Gifford, H. Wymeersch, and M. Z. Win, “NLOS identification and mitigation for localization based on UWB experimental data,” *IEEE J. Selected Areas in Communications*, vol. 28, no. 7, pp. 1026–1035, Sep. 2010.

[11] R. Ali, Y. Liu, S. A. Nawaz, R. C. Malik, and E. S. Lohan, “Tightly coupling fusion of UWB ranging and IMU pedestrian dead reckoning for indoor localization,” *IEEE Access*, vol. 9, pp. 164206–164221, 2021.

[12] Y. Guo, X. Yan, K. Guo, and Q. Zhao, “Combining dilution of precision and Kalman filtering for UWB positioning in a narrow space,” *Remote Sensing*, vol. 14, no. 21, art. 5409, 2022.

[13] Decawave (now Qorvo), *DW1000 User Manual and Datasheet*, Decawave Ltd., Dublin, Ireland.

[14] *IEEE Standard for Local and Metropolitan Area Networks – Part 15.4: LR-WPANs Amendment 1: Alternate PHYs*, IEEE Std 802.15.4a-2007, 2007.

[15] D. Kwak and J.-W. Chong, “A New Double Two-Way Ranging Algorithm for Ranging System,” in *Proc. 2nd IEEE Int. Conf. Network Infrastructure and Digital Content*, 2010, pp. 470–473.

[16] M. Z. Win and R. A. Scholtz, “Impulse Radio: How It Works,” *IEEE Communications Letters*, vol. 2, no. 2, pp. 36–38, 1998.